

Assembly Arithmetic Algorithms

16-bit DOS Edition

```
org 100h
```

```
mov ah,2
```

```
mov dl,20h
```

```
loop_start:
```

```
int 21h
```

```
inc dl
```

```
cmp dl,7Fh
```

```
jne loop_start
```

```
mov ax,4C00h
```

```
int 21h
```

Assembly Arithmetic Algorithms-DOS

16-bit DOS Edition

Chastity White Rose

This book is available at

<https://leanpub.com/assemblyarithmeticalgorithms-dos>

This version was published on 2026-06-11



This is a [Leanpub](#) book. Leanpub empowers authors and publishers with the Lean Publishing process. [Lean Publishing](#) is the act of publishing an in-progress ebook using lightweight tools and many iterations to get reader feedback, pivot until you have the right book and build traction once you do.

© 2026 Chastity White Rose

Contents

Assembly Arithmetic Algorithms	1
Preface	2
Introduction	4
Required Knowledge	4
Low Level	4
Why DOS?	5
Online Example Programs	5
Chapter 1: The First Program	6
Assemble with FASM	6
Assemble with NASM	8
Disassembling the Program	8
The General Purpose Registers	9
Interrupt Information	10
Chapter 2: The putstring Function	12
Chapter 3: The intstr and putint functions	16
Chapter 4: Chastity's Intel Assembly Reference	26
mov	26
add	27
sub	27
mul	28
div	29
cmp	31
jmp	31
push/pop	33
Take it slow	34

Chapter 5: Integer Sequences and Their Application in Learning 35
 Fibonacci numbers 35
 Powers of 2 37
 Prime Numbers 41
 How to use these examples 44

Chapter 6: The strint Function 45
 The Program 45
 The Function 47
 The Output 49

Chapter 7: Translating Assembly to Other Programming Languages . . 50
 main.asm 50
 chastelib16.asm 52
 Assembly Test Suite Output 58
 main.c (The C Test Suite) 63
 chastelib.h (The C chastelib library) 65
 Portable Assembly Language 70

Chapter 8: Going from DOS to Linux or Windows 72
 main.asm (32 bit) 73
 main.asm 64 bit 74
 main.s (GNU Assembler 64 bit) 76

Chapter 9: Bitwise Operations for Advanced Nerds 79
 The Bitwise Operations 79
 Example 0: Fake Add 81
 Example 1: Fake Sub 83

Chapter 10: chastehex: Not just a program, but a philosophy 85
 chex.asm 87
 chex.com 100

Chapter 11: Bonus Content 104
 chastehex main.c 104
 chastelib.h 107

Chapter Z: More Documentation 113

Appendix of System Calls for DOS 114
 D-2100-TERMINATE PROGRAM 114

D-2101-READ CHARACTER	114
D-2102-WRITE CHARACTER	114
D-2109-WRITE STRING	115
D-2139-MKDIR	115
D-213A-RMDIR	115
D-213B-CHDIR	116
D-213C-CREAT	116
D-213D-OPEN	116
D-213E-CLOSE	117
D-213F-READ	117
D-2140-WRITE	118
D-2141-UNLINK	118
D-2142-LSEEK	119
D-214300-GET FILE ATTRIBUTES	119
D-214301-CHMOD	120
D-214C-EXIT	120
D-2159-GET ERROR INFO	121
D-2162-GET PSP ADDRESS	122

Assembly Arithmetic Algorithms

16-bit DOS Edition

Preface

You might be surprised to find a book in the 21st century about programming in Assembly Language on DOS. For many people, DOS seemingly disappeared and became irrelevant in the mid 1990s. Fortunately for me, this was not the case. In fact, I regularly used DOS on old computers that my mother's piano students gave me that they no longer wanted. I had an MS-DOS 3.3 Manual, floppy disks of both 5.5-inch and 3.5-inch sizes, and software like WordPerfect and Edlin that you have probably never heard of. I memorized how to write, copy, rename, and delete files without Microsoft Windows or Linux, both of which I did not have until the 21st century, when I was 14 years old and got my first modern laptop, which had Windows 98 and the ability to restart into DOS mode.

As you might expect, I spent more time in DOS than I did on Windows 98. If it had not been for the battery failure, I probably would have used it much longer than I did. A world of text terminals was my playground, and I was not used to moving a mouse and clicking in a Windows graphical user interface. I used Windows 98 to download DOS games from the internet and to play "Castle of the Winds" (an obscure Norse Mythology game you probably also never heard of).

I tell you all this for context so that you understand why the Magic of older computer systems is still with me even as I write this in the year 2025. Just because MS-DOS and Windows 98 are no longer commonly installed on computers does not mean that the old games or programming styles have disappeared, at least not yet. Thanks to emulators like DOSBox and real operating systems like FreeDOS, it is possible to get programs created 40+ years ago to run on computers today even faster than they ran on the original machines they were designed for.

But I wanted to go a step further and write new programs that could run within an emulator and theoretically a computer made in the 1980s. However, the information is getting harder to find. I want to thank the authors, both dead and alive, who have worked to make sure this information was freely available on the internet. In particular, I would most like to thank Randall Hyde (author of "The Art of Assembly") and Ralf Brown (creator of "Ralf Brown's Interrupt

List”). Without this information, I might have never figured out how to write “Hello World!” in DOS 16-bit Assembly Language.

Therefore, I encourage anyone brave enough to read this book to consider that I am just a nerd that feared this information would be lost forever unless I pass on the information compiled by these genius men who have worked hard to help people learn how to accomplish tasks in Assembly Language for old operating systems that are now only known by those who truly seek to understand how computers work!

Introduction

First, let me introduce this book by telling you what I will teach you. By the end of this book, you will have enough information to write any text-based console program in the form of a 16-bit DOS (Disk Operating System) “.com” file.

The “.com” file was a format used by all versions of MS-DOS, and even supported on Windows up to XP. It has no header information and is limited to 64 kilobytes of memory. Rather than viewing the limitation as a weakness, I view it as a strength because it forces me to be a better programmer and squeeze the most out of every byte.

Required Knowledge

To get the most out of this book, some background on the Binary and Hexadecimal numeral systems is going to be helpful, but this is not strictly required because I will be providing functions you can use in your code that will convert between decimal (base ten), binary (base two), and hexadecimal (base 16).

However, I would say that experience in at least one programming language is necessary for an understanding of terminology like “arrays”, “pointers”, “addresses”, “integers”, “floating point”, etc. I recommend the C Programming Language as a start. C++ is also a good starting language, but it tends to abstract details away that directly apply to Assembly Language, which is the lowest level a human can go for understanding a computer.

Low Level

Low level is a term that confuses people. People think something high-level is better than low-level. In simple terms, humans consider themselves superior to machines and therefore think themselves higher or more important because of their abstract thought.

A computer thinks only in terms of numbers. A computer may not understand “high-level” abstractions such as love, religion, philosophy, etc, but that

is not its job. A computer must add, subtract, multiply, and divide. These are the four arithmetic functions that many humans struggle with.

Therefore, I ask you, between a human and a computer, who is really low level or high level? In the age of Artificial Intelligence taking over human jobs and beating humans at Chess, we would all do well to take this question seriously.

I wrote this book because I think like a machine, and I hope to help others think this way because it is the best way to learn programming and control your computer by writing Assembly Language programs, or to go back to your favorite programming language with a greater understanding of why things work as they do.

Why DOS?

DOS is not at all like Windows or Linux, because it comes from an older time when people were expected to read books, and even video games often came in the form of source code published in books. Therefore, I have decided to dedicate this book to the Disk Operating System, more commonly called DOS, and made famous by MS-DOS, which was Microsoft's version that people in the 80s and 90s remember. Later on, I plan to write a book on programming on Linux using similar but modern methods.

Online Example Programs

Although you can retype every example program from this book and try to run it in your DOS emulator, I also provide the examples as downloads from my Github repository for teaching Assembly.

<https://github.com/chastitywhiterose/Assembly/tree/main/fasm/dos/AAA-DOS-book-examples>

My samples are free and under the “GNU General Public License v3.0” because my intention is to make Assembly Language easy to learn for everyone without any restrictions. My hope is that others find the joy of programming in DOS, no matter whether they learn it from me or whether they learned it from someone else who may have picked up a few things from me.

Chapter 1: The First Program

For this chapter, I will give and explain the source code of an example program that works in DOS, how to assemble it using the tools FASM or NASM, and finally, how the program works line by line.

First, here is the source code of a program that looks like nonsense but does something really cool.

```
1  org 100h
2
3  mov ah,2
4  mov dl,20h
5  loop_start:
6  int 21h
7  inc dl
8  cmp dl,7Fh
9  jne loop_start
10
11 mov ax,4C00h
12 int 21h
```

You will need an assembler. My first recommendation is FASM, the Flat Assembler.

<https://flatassembler.net/>

You can download FASM and install it by putting it in your path. The instructions for doing this depend on your operating system but it can be done on Windows, Linux, or even within a DOS operating system, which you will of course need to run the program.

Assemble with FASM

To assemble this program with FASM, place the source in a file named “main.asm” and run this command

```
fasm main.asm
```

FASM will automatically create a “main.com” file because it understands by the context of “org 100h” that you are intending to create a DOS executable that ends with a “.com” extension. This directive signals that the program starts at address 100 hexadecimal or 256 decimal. This kind of DOS program always starts at that address.

After you have created the “main.com” file, you will need some kind of DOS emulator to run it. I recommend DOSBox because it is easy to set up and has a lot of documentation to help you.

<https://www.dosbox.com/>

As an example of how to use DOSBox efficiently, I have added the path of my working directory where I test my programs directly into my DOSBox configuration file. Instructions for doing this depend on your host operating system. Consult the DOSBox documentation for the location of where it will be on your operating system.

```
1 [autoexec]
2 # Lines in this section will be run at startup.
3 # You can put your MOUNT lines here.
4 mount d ~/git/Chastity-Code-Cookbook/work/
```

This mounts a folder on my Linux system as if it was the D drive recognized by DOS. Back in the DOS and early Windows days, there were “drives” which were all a single letter. A and B were the floppy disk drives, C was the hard disk drive, and sometimes there was a D or E drive for a CDROM drive. DOSBox lets you emulate them and mount any folder on the host operating system (usually Windows or Linux) and access it as you would in DOS.

To switch to the D drive, I just enter

D:

And then I can type “dir” to see the files, and then I can type

main

and the main.com file will run. This works because “.com” and “.exe” files are seen by DOS as a program that can be executed or run.

When you run the program, it will display

```

1  !"#$%&'()*+,-./0123456789:;<=>?@ABCDEFGHIJKLMNOPQRSTUVWXYZ[\]^_`abcdefghijklmnopqrstuvwxyz
   ↳ mnopqrstuvwxyz{|}~

```

Basically the program is displaying every printable character. This is the correct behavior I expected when I wrote the program.

Assemble with NASM

You can assemble the example program with NASM instead of FASM if you wish.

```

1  nasm main.asm -o main.com

```

Disassembling the Program

If you have a disassembler, it is possible to extract the source code from the main.com binary file! I always used “ndisasm” for this because it usually comes installed along with NASM.

```

1  ndisasm -o 0x100 main.com

```

If you disassemble it like this, you will get this as a result:

```

1  00000100  B402          mov ah,0x2
2  00000102  B220          mov dl,0x20
3  00000104  CD21          int 0x21
4  00000106  FEC2          inc dl
5  00000108  80FA7F        cmp dl,0x7f
6  0000010B  75F7          jnz 0x104
7  0000010D  B8004C        mov ax,0x4c00
8  00000110  CD21          int 0x21

```

You will see that it is almost identical to the source except that the loop_start label has been replaced with the number 104h. This is because at the machine code level, there are only numbers.

The first column in the ndisasm output is the address of the instruction. The second are the actual machine code bytes. The third column are the

approximation of the original source code. It has small differences but it is close enough that we can figure out which program it was that was assembled!

Now let me break down why it works by repeating the source but including comments this time

```
1  org 100h          ;tell assembler that code begins running at this address
2
3  mov ah,2          ; move (copy) the number 2 into the ah register
4  mov dl,20h        ; move the number 20 hex=32 dec into the dl register
5  loop_start:       ; the loop starts here
6  int 21h           ; interrupt 21 hex=33 dec for a DOS system call
7  inc dl            ; add 1 to the dl register
8  cmp dl,7Fh        ; compare the dl register with 7F hex = 127 dec
9  jne loop_start    ; Jump if Not Equal to loop_start
10
11 mov ax,4C00h       ; DOS exit call with ah=4C and return al=0
12 int 21h           ; DOS system call to complete the exit function
```

I know you are probably a little bit confused at this point and have many questions such as

- What is an interrupt?
- What is a system call?
- Why do you write your numbers in hexadecimal?
- What is a register?

I would probably give you the wrong definition if I had to explain what an interrupt is from a hardware or software perspective. In this case, the interrupt number 21h is something put into memory by DOS which can be called as if it were a function like you would write in any language.

The reason the interrupts and other numbers are in hexadecimal is because most assembly language books and tutorials use them. Hexadecimal is objectively more convenient because it can be more easily converted to and from binary. For now just remember that interrupt 21h is actually thirty-three and not twenty-one. I have chosen to stick with hexadecimal for this book because it will be relevant later on when I show you other tools which can be used if you understand hexadecimal!

A register is a special variable that exists on a specific CPU type. DOS, Windows, and most Linux operating systems run on an Intel 8086 compatible CPU. I will explain the registers and their functions.

The General Purpose Registers

There are 8 general purpose registers.

- AX: The Accumulator Register
- BX: The Base Register
- CX: The Count Register
- DX: The Data Register
- SI: The Source Index
- DI: the Destination index
- BP: The Base Pointer
- SP: The Stack Pointer

Of those 8 registers, only BX,BP,SI,DI can be used as index variables. This is only a limitation of 16 bit real mode programs like those written in this book. 32-bit and 64-bit programs do not have this limitation, but they have other limitations to worry about and will be covered in future books.

These registers are “general” in that they can do many things, but they each have more “specific” uses also.

In my source code, I use lowercase for the names of instructions and registers, but for this section, I listed them in capital letters because they are actually acronyms named for their purpose according to what Intel had in mind when making the 8086 and above Central Processing Units.

Most of the time, I stick with only AX,BX,CX,DX when writing my programs. If I need extra registers, I will use BP,SI,DI. There are special instructions for them but these are outside the scope of what I am trying to teach with this book.

You might wonder, why isn't there EX,FX,...YX,ZX? Perhaps in a perfect world there should have been, but they probably didn't want to spend the extra money on having extra registers for every 26 letter of the alphabet.

In the next chapter I am going to introduce a program that can print any string you give to it. Basically, it will be the proper “Hello World” program that you were expecting.

Interrupt Information

All code in this book depends on different functions of interrupt 21h. I have provided the following link to where you can read about the most common functions of this interrupt which will be used in this book

<https://github.com/chastitywhiterose/Assembly/blob/main/doc/Chastity-DOS-Interrupts.txt>

The function chosen depends on the value of the AH register (the upper half of the AX register). Depending on which function is selected, then other registers act as options or arguments to these functions. The example I included in this chapter shows only the ah=2 call (equivalent of C putchar call) and the exit call of ah=4Ch with al being the return value.

Chapter 2: The putstring Function

For this next program, I will be introducing the “putstring” function that I wrote. This function takes the address of wherever the ax register points to, then does a routine to scan for the next zero byte. Then it subtracts the original address from the address where the zero was found. By doing this, it knows how many bytes there are to print in the string.

Then it loads the registers in the following way:

- AH = 40h (the DOS write call)
- BX = file handle
- CX = number of bytes to write
- DS:DX -> data to write

Then interrupt 21h is called and this executes the most fun system call possible. It can print any string you give it. Take the following source and assemble it with either FASM or NASM as described in chapter 1.

```
1  org 100h
2
3  main:
4
5  mov ax,text
6  call putstring
7
8  mov ax,4C00h
9  int 21h
10
11 text db 'Hello World! ',0Dh,0Ah,0
12
13 ;This section is for the putstring function I wrote.
14 ;It will print any zero terminated string that register ax points to
15
16 stdout dw 1 ; variable for standard output so that it can theoretically be
   ↳ redirected
17
18 putstring:
19
20 push ax
```

```

21 push bx
22 push cx
23 push dx
24
25 mov bx,ax                ;copy ax to bx for use as index register
26
27 putstring_strlen_start:  ;this loop finds the length of the string as part of
    ↪ the putstring function
28
29 cmp [bx], byte 0         ;compare this byte with 0
30 jz putstring_strlen_end  ;if comparison was zero, jump to loop end because we
    ↪ have found the length
31 inc bx                  ;increment bx (add 1)
32 jmp putstring_strlen_start ;jump to the start of the loop and keep trying until
    ↪ we find a zero
33
34 putstring_strlen_end:
35
36 sub bx,ax                ; sub ax from bx to get the difference for number of
    ↪ bytes
37 mov cx,bx                ; mov bx to cx
38 mov dx,ax                ; dx will have address of string to write
39
40 mov ah,40h               ; select DOS function 40h write
41 mov bx,[stdout]          ; file handle 1=stdout
42 int 21h                  ; call the DOS kernel
43
44 pop dx
45 pop cx
46 pop bx
47 pop ax
48
49 ret

```

If you assembled it and ran it in DOS, you should get

```
1 Hello World!
```

As the result, I know this doesn't seem very impressive, but this program accomplishes a lot. You see, in Assembly, you don't have access to C's "printf" or even "puts". However, the 40h call of DOS is useful enough that during the course of this book, I will introduce how you can use my functions to replace the standard library output functions or even modify them if you don't like the way I wrote them!

If I had to compare DOS 40h to something in C, I would compare it to the

“fwrite” function which writes a specified number of bytes to a specific file stream. Writing to file 1 is the same as writing to the screen.

Specifically, entry “D-2140” in “INTERRUPT.F” of Ralf Brown’s Interrupt list is where I got my documentation I required to write the “putstring” function.

If you look at the source, you will see I included a “main:” label. This wasn’t actually necessary but I added it for clarity and to distinguish the main function from the putstring function. This is a convention I will keep for the remainder of this book.

The “Hello World!” is defined as data in the assembler like this.

```
1 text db 'Hello World!',0Dh,0Ah,0
```

That line is not actually assembly language but is pure data according to the way it is defined in both FASM and NASM. The “0Dh,0Ah” are bytes defining the end of a line in DOS. Finally, I ended the string with a 0 because the putstring function uses it to know when to stop printing.

Therefore, the entire main function is:

```
1 main:
2
3 mov ax,text
4 call putstring
5
6 mov ax,4C00h
7 int 21h
```

The “call” instruction calls a function. As far as assembly is concerned, a function is just a label which can be used either as a function name or used to create the equivalent loops you would normally create with the “while” or “for” loop in the C Programming Language. The difference is that a “ret” instruction will send the program back to where it should be when the function is done. If you forget the “ret” instruction, you will cause a crash because the computer will keep trying to execute code that you did not write. Luckily, if you are running your DOS program in DOSBox, you will only crash the emulator and not your host operating system.

When I designed the putstring function, I chose ax as the register to first hold the address of the string. I did this because ‘a’ is the first letter of the alphabet and so I use it as the first argument for any of my written functions.

However, considering that the dx register is used for the data location in the DOS write call, perhaps it would have made more sense to write it that way. This is just a matter of personal taste and I mention it to show you that even assembly language allows a certain amount of personal style when writing your code.

You may have noticed the push instructions at the beginning of the putstring function and the pop instructions at the end of the function. The push and pop instructions operate the “stack”. The stack is a First In, Last Out method of managing temporary storage.

Because we are required to use those 4 registers for the system call, we back them up and then restore them. This way, the registers retain their original value as if we had never modified them in the function. This may not seem important now, but in the following chapters, we will be printing lots of strings and numbers, so it is important that their values don’t change while we use them in integer sequence programs later on!

But all the putstring function does is print a string of text. It can’t print numbers as humans would expect to see them, at least not yet! In the next chapter, I will correct this problem by showing you a function that can print integers!

If you don’t understand the reason the programs in chapter 1 and 2 work, that’s because I am first establishing a code base which can be used to give you feedback. Without a way of printing output, we have no idea whether our code is correct!

Chapter 3: The intstr and putint functions

In this chapter, I will introduce two new functions designed to work with the putstring function from the last chapter. We can already print a string, but this doesn't work for numbers. Fortunately I have written my own functions which can convert whatever number is in the ax register into a string which can be displayed.

The source of a complete program is below. Take a good look at it even if you don't understand it at first because I will be explaining some things about it.

```
1  org 100h
2
3  main:
4
5  mov ax,text
6  call putstring
7
8  mov [radix],word 10
9  mov [int_width],word 1
10
11 mov ax,1
12
13 main_loop:
14 call putint
15 add ax,ax
16 cmp ax,0
17 jnz main_loop
18
19 mov ax,4C00h
20 int 21h
21
22 text db 'This program displays numbers!','0Dh,0Ah,0
23
24 ;This section is for the putstring function I wrote.
25 ;It will print any zero terminated string that register ax points to
26
27 stdout dw 1 ; variable for standard output so that it can theoretically be
   ↪ redirected
```

```

28
29 putstring:
30
31 push ax
32 push bx
33 push cx
34 push dx
35
36 mov bx,ax                ;copy ax to bx for use as index register
37
38 putstring_strlen_start:  ;this loop finds the length of the string as part of
    ↪ the putstring function
39
40 cmp [bx], byte 0         ;compare this byte with 0
41 jz putstring_strlen_end  ;if comparison was zero, jump to loop end because we
    ↪ have found the length
42 inc bx                  ;increment bx (add 1)
43 jmp putstring_strlen_start ;jump to the start of the loop and keep trying until
    ↪ we find a zero
44
45 putstring_strlen_end:
46
47 sub bx,ax                ; sub ax from bx to get the difference for number of
    ↪ bytes
48 mov cx,bx                ; mov bx to cx
49 mov dx,ax                ; dx will have address of string to write
50
51 mov ah,40h               ; select DOS function 40h write
52 mov bx,[stdout]          ; file handle 1=stdout
53 int 21h                 ; call the DOS kernel
54
55 pop dx
56 pop cx
57 pop bx
58 pop ax
59
60 ret
61
62 ;this is the location in memory where digits are written to by the intstr
    ↪ function
63 int_string db 16 dup '?' ;enough bytes to hold maximum size 16-bit binary
    ↪ integer
64 ;this is the end of the integer string optional line feed and terminating zero
65 ;clever use of this label can change the ending to be a different character
    ↪ when needed
66 int_newline db 0Dh,0Ah,0 ;the proper way to end a line in DOS/Windows
67
68 radix dw 2 ;radix or base for integer output. 2=binary, 8=octal, 10=decimal,
    ↪ 16=hexadecimal
69 int_width dw 8

```

```
70
71  intstr:
72
73  mov bx,int_newline-1 ;find address of lowest digit(just before the newline 0Ah)
74  mov cx,1
75
76  digits_start:
77
78  mov dx,0;
79  div word [radix]
80  cmp dx,10
81  jb decimal_digit
82  jnb hexadecimal_digit
83
84  decimal_digit: ;we go here if it is only a digit 0 to 9
85  add dx,'0'
86  jmp save_digit
87
88  hexadecimal_digit:
89  sub dx,10
90  add dx,'A'
91
92  save_digit:
93
94  mov [bx],dl
95  cmp ax,0
96  jz intstr_end
97  dec bx
98  inc cx
99  jmp digits_start
100
101  intstr_end:
102
103  prefix_zeros:
104  cmp cx,[int_width]
105  jnb end_zeros
106  dec bx
107  mov [bx],byte '0'
108  inc cx
109  jmp prefix_zeros
110  end_zeros:
111
112  mov ax,bx ; store string in ax for display later
113
114  ret
115
116  ;function to print string form of whatever integer is in ax
117  ;The radix determines which number base the string form takes.
118  ;Anything from 2 to 36 is a valid radix
```

```
119 ;in practice though, only bases 2,8,10,and 16 will make sense to other
    ↳ programmers
120 ;this function does not process anything by itself but calls the combination of
    ↳ my other
121 ;functions in the order I intended them to be used.
122
123 putint:
124
125 push ax
126 push bx
127 push cx
128 push dx
129
130 call intstr
131 call putstring
132
133 pop dx
134 pop cx
135 pop bx
136 pop ax
137
138 ret
```

If you assemble and run this program, you will get the following output.

```
1 This program displays numbers!
2 1
3 2
4 4
5 8
6 16
7 32
8 64
9 128
10 256
11 512
12 1024
13 2048
14 4096
15 8192
16 16384
17 32768
```

The program prints ax, adds ax to itself, and then stops as soon as ax “overflows” by going higher than the 16 bits limit. When this happens, it will become zero. Our jnz means Jump if Not Zero to the main loop.

If you look at the main function you will see that I set the radix to 10 with a mov instruction, even though it defaulted to 2. This is because most humans are used to decimal, AKA base ten. The base can be changed at any time in the program however you like.

Another thing you will notice is that the “putint” function does not process anything at all. It simply backs up the registers, calls the intstr function to create a string and then calls putstring to display it. In this example, a newline is automatically added for convenience. In my own code, I usually have this done manually by another small function, but for the purposes of this book, this default behavior is fine.

The real power of this program is the intstr function and why it works as it does. I will spend the rest of this chapter explaining why it works, why I designed it this way, and why this function is essential for Assembly language programs to make sense at all.

First, before the function begins, I have defined data using db and dw directives that FASM and NASM both understand.

```
1 int_string db 16 dup '?'
```

This creates sixteen bytes of question marks. The actual bytes used here don't matter but I used '?' marks to signal that the data that will go here is unknown when the program starts. The actual digits of the number we convert from the ax register will replace these bytes when we call the intstr function.

```
1 int_newline db 0Dh,0Ah,0
```

This line takes care of two problems. First, it makes sure that there is a zero byte after the 16 bytes of data from the int_string variable. It also includes the bytes thirteen and ten in hexadecimal notation. This is how newlines are saved if you have used a text editor in DOS or Windows. When you hit the return key it generates both these bytes to define that a line of text has ended. If you were to change the 0Dh to 0, then the program would print the numbers but without separating them with lines or even spaces. Such a thing would make the numbers hard to read. That is why the default behavior is to print a number and end the line for readability. This works for most simple integer sequence programs I will include in this book.

```
1 radix dw 2
2 int_width dw 8
```

These are two variables that define the base/radix of the number string generated and also the “width” AKA how many leading zeroes are used when writing it. The width should be set to one for most programs when decimal integers are expected. However, setting the width to 8 or 16 makes sense for binary integers where seeing the exact bits in their positions lined up is essential.

The defaults I have chosen include radix 2 (the binary numeral system) and a width of 8 (for seeing 8 bits of a byte). But the defaults are irrelevant for what you need to know. See how the main function in my example program for this chapter overwrites them in the main function.

```
1 intstr:
```

This is the label defining the start of the intstr function. If this label were not present, then the “call putint” statement would not know what you mean. Also, keep in mind that “intstr” is just an address in memory much like “radix” and “int_width” are addresses that tell where bytes of data are. However, the convention I use is that labels ending with a colon are labels that will be called with the “call” instruction or jumped to with a jmp or j?? instruction. There will be more explanation of conditional jumps in chapter 4.

```
1 mov bx,int_newline-1
2 mov cx,1
```

Before the loop in the intstr function, we set bx equal to the address of the byte before int_newline. This will be the final ‘?’ we defined earlier. The cx register is set to one to signal the number of bytes that will exist in the string. Every number, including 0 and 1 have at least one digit no matter which base you use. The cx register will come into use near the end of the function in its own loop.

```
1 digits_start:
```

This is a label defining the loop of where the digits are generated in the string.

```
1  mov dx,0;
2  div word [radix]
3  cmp dx,10
4  jb decimal_digit
5  jnb hexadecimal_digit
```

`dx` is set to 0 because this has to be done before the “`div`” instruction. Otherwise, it will be mistaken as part of the dividend. This is a quirk of the x86 family of CPUs. The `div` instruction takes one argument, in this a word value from address `radix` and divides the `ax` register. If we don’t zero `dx`, it will use the `dx` register as an upper 16 bits of the number we are dividing from as well as using `ax` as the lower 16 bits.

For a full explanation of this division behavior, see section “2.1.3 Binary arithmetic instructions” in the FASM documentation. Tomasz Grysztar explains it better than I can and his information greatly helped me when trying to figure out why my function wasn’t working.

After the division, the `dx` register contains the remainder of the division. The `ax` register will be whatever it was divided by the `radix`. Knowing this, we “`cmp dx,10`” which means compare the `dx` register with 10. If it is less or below 10, then we know it is a decimal digit in the range of 0 to 9. If it is greater than or equal. Based on these conditions, we jump to one of two sections. One handles decimal digits and the other handles hexadecimal digits. Technically bases 2 to 36 are handled by my program as a consequence of the way I wrote it, but I wrote it with the idea that I would be using this function with only 3 different bases.

- base 2 or binary for my personal enjoyment
- base 16 or hexadecimal for a short form of binary
- base 10 or decimal which is what humans know how to read. It will be used mostly in this book

```
1 decimal_digit: ;we go here if it is only a digit 0 to 9
2 add dx,'0'
3 jmp save_digit
4
5 hexadecimal_digit:
6 sub dx,10
7 add dx,'A'
```

These two sections do the math of converting the byte digit into a character in ASCII representation that is printable. In either case, code moves on to the save_digit label after these.

```
1 save_digit:
2
3 mov [bx],dl
4 cmp ax,0
5 jz intstr_end
6 dec bx
7 inc cx
8 jmp digits_start
9
10 intstr_end:
```

This tiny section saves the digit we obtained from this pass of the loop. The dl register is the lower byte of the dx register so we store this character of the digit into the address pointed to by bx.

Keep in mind that pointers are a primary feature in Assembly Language despite being criticized in C/C++ and excluded entirely from other languages like Java.

Next, we compare ax with zero. If it is zero, there are no more digits to write and we will end this loop by jumping to “intstr_end”. Otherwise we decrement (subtract 1 from bx) so that it will point to the digit left of the one we saved this time. We also increment cx so that it knows at least one more digit is to be written because the loop will happen again. We unconditionally jump to digits_start to process digits and save them until ax equals zero.

After ax is zero, we still have one more job to do in this function. The following loop will prefix the string with extra ‘0’ characters while cx is less than the int_width variable. This will be important for those who need the digits lined up to their place values. This is much more important for binary and hexadecimal than it is decimal, but it can still be helpful in decimal as I will show in a later chapter.

```
1  prefix_zeros:
2  cmp cx,[int_width]
3  jnb end_zeros
4  dec bx
5  mov [bx],byte '0'
6  inc cx
7  jmp prefix_zeros
8  end_zeros:
```

There is only one more instruction before we return from this function. We copy the `bx` register to the `ax` register because it points to the beginning of the string. This means that my `putstring` function will accurately display it because it expects `ax` to contain the address of the string.

```
1  mov ax,bx
```

Last but not least, we still have to end the function by returning to the caller.

```
1  ret
```

Before I end this chapter, I want to explain why I chose the register `ax` as the foundation for the behavior of my Assembly functions. `ax` is a special register in the sense that multiplication and division instructions use it as the required number we are multiplying or dividing. The Intel architecture treats this register as being more important for this reason.

But it is not just that, the programmers of DOS decided that the `ax` register was what decided which function of interrupt 21h would be called.

Therefore, because others already treated register `ax` as special, and since ‘a’ is the first letter of the alphabet, I decided that it would be the foundation of all my functions in “chastelib”, my DOS Assembly Standard Library. You are not expected to take my functions as the way things must be done. Once you are done with this book, you may continue learning beyond my skills and may decide that using another register makes more sense than `ax`.

I wrote this book to teach Assembly Language as I understand it, not to force my coding practices on you. However, I add these extra details so that other programmers who have experience in Assembly will have answers before they start emailing me: ***“Chastity, why didn’t you write the function this way!”***

You can save a few bytes if you use instruction ??? instead or you could achieve faster speed if you avoided this jump here.”

I am letting you know now, I wrote the code for simplicity rather than performance. I use a very limited subset of the instructions available to the Intel 8086 family of CPUs. I firmly believe that all math for programs I want to write can be written using only **`mov,push,pop,add,sub,mul,div,cmp,jmp`** (and **conditional jumps as well**).

Chapter 4: Chastity's Intel Assembly Reference

I use a very small subset of the Intel 8086 family instruction set. This is both because I want to limit it to my small memory (my brain memory, not computer memory) and because I only care about instructions that existed on CPUs at the time of DOS operating systems. Entire video games and operating systems were written either in Assembly or in C programs that were translated to Assembly. Newer CPUs introduced more instructions but I would argue that these were for convenience or higher speed in limited cases.

For portability, I stick with the instructions I will teach you in this chapter. By portability, I mean portable between as many CPUs of the intel family. Other processors are of course incompatible but they have their own equivalents by different names.

Important note. All program listings in this chapter assume that you also included the putstring,intstr,and putint functions as shown in chapters 2 and 3. This can be done by including external files or just copy pasting their text after the system exit call from ax=4C00 and interrupt 21h.

mov

The mov instruction copies a number from one location to another. In the FASM and NASM assemblers, the instruction always takes the form

```
mov destination,source
```

Think of it as “destination=source” as you would write in C. For example, in the following program which prints the number 8, we see that most of the required data is set up with mov instructions.

```
1  org 100h
2  main:
3
4  mov word [radix],10
5  mov word [int_width],1
6
7  mov ax,3
8  mov bx,5
9  add ax,bx
10
11 call putint
12
13 mov ax,4C00h
14 int 21h
```

That program also contains the `call`, `int`, and `add` instructions to make a program that does something useful. However, `mov` instructions take up the largest part of any program. Whether you are filling a register with a number, another register, or a memory location, the `mov` instruction is the way to do it.

add

Next to `mov`, you will see that `add` is going to be your friend in Assembly a lot. In the previous example, we saw that 3 and 5 were added to make 8. Just like `mov`, `add` follows the same rules.

`add destination,source`

- Source is left of Destination and separated by a comma
- Source and destination can be registers and memory locations
- Source and Destination cannot both be memory location

Most instructions that take two arguments follow these same rules. Once you have mastered `mov` and `add`, you can handle almost anything in a program because you know the basic rules.

There is also the “`inc`” instruction which takes only one item and adds 1 to it. This is just a shorter way of saying “`add Destination,1`”

sub

As its name implies, sub will subtract the Source from the Destination.

```
sub destination,source
```

Since it follows the same rules as mov and add (starting to see a pattern yet?), subtraction is just as easy as addition.

Just as “add” has “inc”, “sub” has the “dec” instruction which subtracts 1. Adding or subtracting 1 are probably the most common thing ever done while programming in any language.

Just as a review of the mov,add,sub instructions, here is a small program to show their effect.

```
1  org 100h
2  main:
3
4  mov word [radix],10
5  mov word [int_width],1
6
7  mov ax,8
8  call putint
9  add ax,ax
10 call putint
11 sub ax,4
12 call putint
13
14 mov ax,4C00h
15 int 21h
```

That program will output the following.

```
1  8
2  16
3  12
```

This is because we set ax to 8, then we added ax to itself, and finally we subtracted 4 from ax. Once you think about how easy this is, read on to see how multiplication and division work.

mul

The `mul` instruction is slightly different than The previous instructions. It takes only one operand which must be either a register or memory location. It multiplies `ax` by the value of this operand. If the value is too large to fit within the `ax` register, it puts the higher bits into `dx`.

div

The `div` instruction divides `ax` by the operand you give it (the divisor). However, division is a tricky operation because not every number divides evenly into another. It is also more complicated by the fact that the `dx` register is assumed to be the upper half of the bits in the dividend while `ax` is the lower bits of the dividend.

I know it sounds complicated but it is easier than I can explain. I can illustrate this with a small program that multiplies and divides!

```
1  org 100h
2  main:
3
4  mov word [radix],10
5  mov word [int_width],1
6
7  mov ax,12
8  call putint
9  mov bx,5
10 mul bx
11 call putint
12 mov bx,8
13 mov dx,0
14 div bx
15 call putint
16 mov ax,dx
17 call putint
18
19 mov ax,4C00h
20 int 21h
```

The output of that program is this:

```
1 12
2 60
3 7
4 4
```

This is because 12 was multiplied by 5 to get 60. Then we attempted to divide 60 by 8. It goes in only 7 times (which equals 56). This means the remainder is 4, which is stored in the dx register after the division.

You may also notice in the source above that I set dx to zero before the div instruction. If this is not done, the dx might have mistakenly had another number and been interpreted as part of the dividend.

I also think some terminology about division is helpful here.

- Dividend: The number we are dividing from.
- Divisor: The number we are dividing the dividend by. How many times does this number subtract from the dividend?
- Quotient: The result of the division.
- Remainder: What is left over if the divisor could not divide perfectly into the dividend.

As much as I love math, I find some of these terms confusing when I try to explain them in English. Let's face it, I am better at Assembly Language and the C Programming Language than I am with English, but it looks like you're stuck with me because normal people are not autistic enough to care!

Division is repeated subtraction, just as multiplication is repeated addition. Most of the time, modern CPU hardware can multiply and divide faster by the mul and div instructions than if you have manually made loops of repeating addition or subtraction, but the result would have been the same.

6502 CPUs (used for the Nintendo Entertainment System) did not have multiplication and division instructions, and so programmers really did use loops of adding and subtracting to get the results needed. On Intel CPUs, mul and div are available, and I suggest using them, but please don't forget what these operations really are.

For a more in depth explanation of the mul and div instructions, I will include those written by Tomasz Grysztar (creator of the FASM assembler) in the official "flat assembler 1.73 Programmer's Manual"

mul performs an unsigned multiplication of the operand and the accumulator. If the operand is a byte, the processor multiplies it by the contents of AL and returns the 16-bit result to AH and AL. If the operand is a word, the processor multiplies it by the contents of AX and returns the 32-bit result to DX and AX.

div performs an unsigned division of the accumulator by the operand. The dividend (the accumulator) is twice the size of the divisor (the operand), the quotient and remainder have the same size as the divisor. If divisor is byte, the dividend is taken from AX register, the quotient is stored in AL and the remainder is stored in AH. If divisor is word, the upper half of dividend is taken from DX, the lower half of dividend is taken from AX, the quotient is stored in AX and the remainder is stored in DX.

* * *

Perhaps you can see that Assembly language is nothing more than a fancy calculator, except better. This is because there is no question which order the operations take place in. There is no need for mnemonics like “Please excuse my dear Aunt Sally” to remind us “Parentheses, Exponents, Multiplication and Division (from left to right), and Addition and Subtraction”.

There are still two more instructions before we can construct useful programs. In fact, my previous examples have used these already, but now it is time to explain them in depth.

cmp

The *cmp* instruction compares two operands but does not do any math with them. They remain unchanged but modify flags in the processor that allow us to jump based on certain conditions.

jmp

The *jmp* instruction jumps to another location regardless of any conditions. It has a family of other jump instructions that jump only if certain conditions are true. In fact many of them have multiple names for the same operation. For

example `je` and `jz` both jump if the two numbers compared would be zero if they were subtracted. This would only be true if they are the same.

Here is a small chart, but it does not cover every alias for these.

Instruction	Meaning
<code>je/jz</code>	jump if equal
<code>ja</code>	jump if above
<code>jb</code>	jump if below
<code>jne/jnz</code>	jump if not equal
<code>jna</code>	jump if not above
<code>jnb</code>	jump if not below

Aside from those main 6 conditional jumps that I have memorized, there also exists `jl`(jump if less) and `jg`(jump if greater). However, they are for signed/negative numbers which I have not covered. Personally I don't agree with the way negative numbers are represented in computers but I know that understanding the context of signed vs unsigned is important for more complex programs. Once again, I recommend the FASM programmers manual for details that I have excluded for the purpose of keeping this book short.

The following program can print a message telling you whether `ax` is less than , equal to, or more than `bx`. Upon this foundation all the conditional jumps in my programs and functions are based.

```
1  org 100h
2  main:
3
4  mov word [radix],10
5  mov word [int_width],1
6
7  mov ax,5
8  mov bx,8
9  cmp ax,bx
10 jb less
11 je same
12 ja more
13
14 less:
15 mov ax,string_less
16 jmp end
```

```
17  same:
18  mov ax,string_same
19  jmp end
20  more:
21  mov ax,string_more
22  jmp end
23
24  end:
25  call putstring
26
27  mov ax,4C00h
28  int 21h
29
30  string_less db 'ax is less than bx',0
31  string_same db 'ax is the same as bx',0
32  string_more db 'ax is more than bx',0
```

Personally, I think that the system of conditional jumps makes a lot of sense. Other programming languages such as BASIC and C have “goto” statements that work like this. For example, `if(ax<bx){goto less;}`.

The only thing I have found difficult is remembering which acronym means which condition. However, since I created the chart in this chapter, now I can refer to it and you can too! As long as I keep these main six types of conditions in my head, and am working with unsigned numbers, I can write almost any assembly program from scratch.

push/pop

The push and pop instructions are something you have already seen in my code. They operate on what is called the “stack”. Basically, when you push something, it is like pushing a box of cereal onto a shelf at Walmart. The last item pushed is at the front and will be the first item a customer sees. This is what is called a Last In First Out.

Not only is the stack useful for saving the value of registers temporarily as I do, but without it, it would not be possible to have callable functions. When you call a function with “call”, it is the same as a “jmp” to that location except that it pushes the address where the program was before the call. The “ret” instruction returns to the location that called the function and then proceeds to instructions after it.

The sp register, as I mentioned in chapter 1, is the stack pointer. Every time you push a value, it stores it at the address the stack pointer is pointing to and

then subtracts the size of the native word size. For example, this is always 16 bits in the context of DOS programming for 16 bit .com files. This means that you can use it with the other registers to save their value for later.

In the next chapter, I will show a useful example of the push and pop instructions and explain a little bit more about this.

Take it slow

I know I hit you with a lot of information in this chapter, but trust me, I am intentionally leaving out a lot because I don't want this book to be the size of the Intel® 64 and IA-32 Architectures Software Developer Manuals.

<https://www.intel.com/content/www/us/en/developer/articles/technical/intel-sdm.html>

There are hundreds of instructions for Intel machines and yet if you combine the instructions I have described in this chapter with the “call”, “int”, and “ret” instructions required for calling functions for input and output, you will see that it is possible to write almost any program I want with these instructions.

I am sharing what I have learned from reading the Intel Manuals and the API references available for DOS so that you don't have to spend as much time figuring these things out as I did. What I can tell you, though, is that the result was worth it because I have been able to write programs to accomplish tasks faster than my C programs could. At the same time, the Assembly versions took longer to write than the C versions did. This is the price I must pay to have high performing code.

Also, there are some bitwise instructions by the names of AND, OR, XOR, NOT, SHL, SHR that are sometimes useful for making programs faster and smaller. However, these only make sense in the context of the Binary Numeral System and I suspect that the average reader of this book does not have the 25 years of experience in Binary math that I do.

I will be explaining more about these operations in a later chapter because they help a lot when trying to optimize programs for size and speed. However they can make programming LOOK complicated and scare away potentially great new programmers who are just trying to learn to apply the 4 regular arithmetic operations of addition, subtraction, multiplication, and division which apply to all number bases.

Chapter 5: Integer Sequences and Their Application in Learning

To be a programmer in any language, a person needs more than information. There must be a motivation for something you want to make. The challenge is that when you are a beginner, it can be easy to get discouraged because you won't be making anything big or impressive to other people at the start. All you have to work with in most programming languages is displaying text and numbers.

Later on, you can learn to use third-party libraries or native APIs for your operating system. However, what I have always disliked is that the internals of how they work are hidden or obfuscated so that you don't know how they work.

But if you love math as I do, you will never have a problem testing your ability by writing small programs to print integer sequences. In this chapter, I will be sharing 3 of my favorite sequences.

- [Fibonacci numbers](#)
- [Powers of 2](#)
- [Prime Numbers](#)

If you have the ebook edition of this book, you will be able to click the links above and learn more about these sequences. Either way, I will show you the code that makes printing these sequences easy. even in Assembly Language

Fibonacci numbers


```
1  org 100h
2  main:
3
4  mov word [radix],10
5  mov word [int_width],1
6
7  mov ax,0
8  mov bx,1
9
10 Fibonacci:
11
12 call putint
13 add ax,bx
14 push ax
15 mov ax,bx
16 pop bx
17 cmp ax,1000
18 jb Fibonacci
19
20 mov ax,4C00h
21 int 21h
22
23 include 'chastelib16.asm'
```

When executed, that program will output the following sequence

```
1  0
2  1
3  1
4  2
5  3
6  5
7  8
8  13
9  21
10 34
11 55
12 89
13 144
14 233
15 377
16 610
17 987
```

Just by looking at it, hopefully you see the pattern. Each number is the sum of the previous two numbers. The loop in this program needed to swap the numbers in ax and bx each time before the next add. Technically, there are

two other ways I could have achieved it. I could have used `ecx` as a temporary storage. I also could have used the `xchg` instruction, which does the same thing, but the stack provided me a convenient way of doing it. We only needed to save the `ax` register before it was overwritten with `bx`, then we popped into `bx` the pushed `ax` from earlier. None of these methods is more correct than any other, but I tend to favor simplicity and therefore am limiting the type of instructions I use. I also think that using a third register or even another memory location is acceptable for something like this. Still, since the stack is already used for function calls and is an expected part of Assembly, there is no reason not to use it, especially when we only need to save one register.

Powers of 2

```
1  org 100h
2  main:
3
4  mov word [radix],10
5  mov word [int_width],1
6  mov [int_newline],0
7
8  mov cx,0
9
10 mov [array],byte 1
11
12 powers_of_two:
13
14 ;this section prints the digits
15 mov bx,[length]
16 array_print:
17 dec bx
18 mov ax,0
19 mov al,[array+bx]
20 call putint
21 cmp bx,0
22 jnz array_print
23 call putline
24
25 ;this section adds the digits
26 mov dl,0
27 mov bx,0
28 array_add:
29 mov ax,0
30 mov al,[array+bx]
31 add al,al
32 add al,dl
```

```
33  mov dl,0
34  cmp al,10
35  jb less_than_ten
36
37  sub al,10
38  mov dl,1
39
40  less_than_ten:
41  mov [array+bx],al
42  inc bx
43  cmp bx,[length]
44  jnz array_add
45
46  cmp dl,0
47  jz carry_is_zero
48
49  mov [array+bx],1
50  inc [length]
51
52  carry_is_zero:
53
54  ;keeps track of how many times the loop has run
55  add cx,1
56  cmp cx,64
57  jna powers_of_two
58
59  mov ax,4C00h
60  int 21h
61
62  length dw 1
63  array db 32 dup 0
64
65  include 'chastelib16.asm'
```

The above program will display the powers of two. This sequence is 1,2,4,8,16,32,64,128,256, etc., all the way until 18446744073709551616, which is two to the 64th power. You will notice that this is far beyond 32768, which was as high as the powers of two program from chapter 3 was. Normally, we could never achieve this in 16-bit DOS mode if we were dealing with native 16-bit integers. Fortunately, there is a method called Arbitrary Precision Arithmetic.

As fancy as that sounds, it is really just using an array of bytes as if they were decimal digits. These lines define variables for the array and the length of the array.

```
1  length dw 1
2  array db 32 dup 0
```

The “length” starts at 1, and the total size of the array is 32 bytes, initialized to zero. These variables form the boundaries of several different loops in the program. This loop is what displays the current used parts of the array.

```
1  ;this section prints the digits
2  mov bx,[length]
3  array_print:
4  dec bx
5  mov ax,0
6  mov al,[array+bx]
7  call putint
8  cmp bx,0
9  jnz array_print
10 call putline
```

The bx register is set to the current value of length, which will be 1 at the start of the program. Then, bx is decremented so that it is 1 less than the length. Remember, arrays range from 0 to the length minus 1 in Assembly, just like would be the case in C and other languages.

During this loop, while bx is not 0, we set ax to 0 and then load al (the lower half of ax), with the byte at the address of the array plus the number in the bx register. We call the putint function on this value to print the number in al.

The newline is not printed during this loop because of the following line near the beginning.

```
1  mov [int_newline],0
```

By turning the newlines off that putint would normally print, we gain control of exactly when we want to. I created another small function named “putline” which prints a newline when I call it. Here is the source code of the putline function.

```
1 line db 0Dh,0Ah,0
2
3 putline:
4 push ax
5 mov ax,line
6 call putstring
7 pop ax
8 ret
```

After the currently used digits in the array are printed, another loop begins that adds the digits to themselves. Each one is loaded into al, then al is added to itself. The dl register, which is initialized to 0, is the “carry” variable. If the result of al+al is less than 10, we jump to the “less_than_ten:” label and write the new digit back to the array in that index. If, however, the digit in al is 10 or above, we have to subtract 10 and then set dl (our carry) to 1 so that the next digit we add to itself will also have the carry added to it.

```
1 ;this section adds the digits
2 mov dl,0
3 mov bx,0
4 array_add:
5 mov ax,0
6 mov al,[array+bx]
7 add al,al
8 add al,dl
9 mov dl,0
10 cmp al,10
11 jb less_than_ten
12
13 sub al,10
14 mov dl,1
15
16 less_than_ten:
17 mov [array+bx],al
18 inc bx
19 cmp bx,[length]
20 jnz array_add
```

The process of this loop is basically the same way we would add the digits of numbers on paper. However, since we are adding the number to itself, the process is greatly simplified.

But perhaps the final piece of this powers of two program that needs a special mention is the part that expands how many digits are displayed by incrementing the length if a carry of 1 still remains. If the final digit processed

had a result of 10 or greater, the carry in `dl` would have been set to 1, but there would not be a digit to add this to.

```
1  cmp dl,0
2  jz  carry_is_zero
3
4  mov [array+bx],1
5  inc [length]
6
7  carry_is_zero:
```

We set the byte at “array+bx” to 1 and then increment the length variable so that the new digit becomes permanently part of the list of bytes that is displayed and added to itself, plus the carry from each previous digit addition.

The total number of bytes declared for the array in the program was 32 with the statement “array db 32 dup 0”. So the loop would stop working if we went beyond this limit. However, it is reasonable to say that if we wanted to, we could get away with reassembling with 30,000 bytes and display powers of two with that many digits. We would still be far under the limit of the 64 kilobyte memory limit for a “.com” program in DOS.

I hope I haven’t lost you with my explanation of the arbitrary precision Powers of 2 program. The original version was written in my first programming language, QBASIC, and was the first time I had successfully learned how to use arrays.

At 14 years old, I was learning the concepts of arrays and memory addresses for the first time. I remember a very helpful user on the Network54 QBASIC forum explained it over and over again until I understood.

The syntax of the Assembly version of the Powers of 2 algorithm may look strange. Still, it follows all the same steps as the original QBASIC program and the C version, which later became part of “Chastity’s Code Cookbook”.

Prime Numbers

```
1  org 100h
2  main:
3
4  mov word [radix],10
5  mov word [int_width],1
6  mov [int_newline],0
7
8  ;the only even prime is 2
9  mov ax,2
10 call putint
11 call putspace
12
13 ;fill array with zeros up to length
14 mov bx,0
15 array_zero:
16 mov [array+bx],0
17 inc bx
18 cmp bx,length
19 jb array_zero
20
21 ;start by filtering multiples of first odd prime: 3
22 mov ax,3
23
24 primes:
25
26 ;print this number because it is prime
27 call putint
28 call putspace
29
30 mov bx,ax ;mov ax to bx as our array index variable
31 mov cx,ax ;mov ax to cx
32 add cx,cx ;add cx to itself
33
34 sieve:
35 mov [array+bx],1 ;mark element as multiple of prime
36 add bx,cx ;check only multiples of prime times 2 to exclude even numbers
37 cmp bx,length
38 jb sieve
39
40 ;check odd numbers until we find unused one not marked as multiple of prime
41 mov bx,ax
42 next_odd:
43 add bx,2
44 cmp [array+bx],0
45 jz prime_found
46 cmp bx,length
47 jb next_odd
48 prime_found:
49
50 ;get next prime read to print in ax
```

```
51  mov ax,bx
52  cmp ax,length
53  jb  primes
54
55  mov ax,4C00h
56  int 21h
57
58  include 'chastelib16.asm'
59
60  length=1000
61  array rb length
```

The primes program uses a method called the “Sieve of Eratosthenes”. It is an ancient but very fast algorithm to implement in almost any programming language. The program can find all primes less than 1000 in less than a second.

A Sieve is a process of elimination. Imagine you have all the numbers from 0 to 100. A prime number, by definition, has only two factors: itself and 1.

The only even prime number is 2. It is the first prime number because 1 times 2 equals 2. There are only two factors. Since all even numbers like 4,6,8,10,12, etc are multiples of 2. We exclude them from the list of possible primes. The new number, which is not crossed out, is 3. We then cross out all multiples of 3. Then the next number still in the list after 3 is 5. 4 doesn't exist because we already excluded multiples of 2. 5 is our next prime number after 3 for this reason. We cross out all multiples of 5 like 10,15,20,25, etc. In fact, some of these would have already been excluded because they are multiples of 2.

In summary, the primes program has an array of 1000 bytes. We use each of these bytes as items to keep track of whether they are prime or not. Every item in the array starts as 0 (prime until proven otherwise). We print 2 because it is a known even prime. We then do the same for 3 because it is the first odd prime. Then we mark all indexes which are a multiple of 3 as 1(not prime). We then skip ahead to the next odd index, which is not marked.

The result will be 2 3 5 7 11 13 17 19 23 29 31 37 41 43 47 53 59 61. The list contains all odd numbers after 2, and some of them are excluded because they are multiples of other odd numbers.

9 is not prime because 33 is 9. 21 is *not prime* because 37 is 21. And so it continues.

This prime algorithm requires a lot of memory, and so finding the first billion primes is not something that can be done in a 64 KB DOS program because of

memory limitations. However, this method is fast because it uses only addition and subtraction (excluding the division used in the `intstr` function of my library). On a modern PC running Linux instead of DOS, it is easier to allocate gigabytes of memory and find lists of even higher primes.

How to use these examples

My suggestion is that you download the examples in this chapter from my github repository rather than trying to type them by hand or copy past them. That way you can assemble them with FASM and run them in the DOSBox emulator to see how they work.

<https://github.com/chastitywhiterose/Assembly/tree/main/fasm/dos/AAA-DOS-book-examples/>

These programs can produce long lists of numbers and so I can't include all the output in this book. You will have to run them to get the full picture of how magnificent they are!

Chapter 6: The strint Function

In this chapter, I will only be introducing one program that uses the three previous functions I described in earlier parts of this book but also includes one more important one!

What this program does is really quite simple, it takes a string of binary integers called “test_int” and converts it into a real integer using a new function called “strint”.

Below is the source of this program. Take a minute to look it over. Afterwards, I will explain more about the “strint” function and how it interacts with the other three functions. “putstring”, “intstr”, and “putint”.

The Program

```
1  org 100h
2
3  main:
4
5  mov ax,main_string
6  call putstring
7
8  mov word [radix],2 ; choose radix for integer input/output
9  mov word [int_width],1
10
11 mov ax,test_int
12 call strint
13
14 mov bx,ax
15
16 mov ax,str_bin
17 call putstring
18 mov ax,bx
19 mov word [radix],2
20 call putint
21
22 mov ax,str_hex
23 call putstring
24 mov ax,bx
25 mov word [radix],16
```

```

26  call putint
27
28  mov ax,str_dec
29  call putstring
30  mov ax,bx
31  mov word [radix],10
32  call putint
33
34  mov ax,4C00h
35  int 21h
36
37  main_string db "This is the year I was born",0Dh,0Ah,0
38
39  ;test string of integer for input
40  test_int db '11111000011',0
41
42  str_bin db 'binary: ',0
43  str_hex db 'hexadecimal: ',0
44  str_dec db 'decimal: ',0
45
46  include 'chastelib16.asm'

```

For a quick review, the 3 previous function do the following.

- putstring: prints a zero terminated string pointed to by ax register
- intstr: converts the integer in ax register into a zero terminated string and then points ax to that string for compatibility with putstring
- putint: calls intstr and then putstring to display whatever number ax equals

And now I introduce to you the final function of my 4 function library that I call “chastelib”.

This function is called “strint” and its importance cannot be overstated. It does the opposite of the “intstr” function. Instead of converting an integer to a string, it does the opposite and takes the string pointed to by ax and converts it into a number returned in the ax register. Much like “intstr”, it uses the global [radix] variable to know which base is being used.

But the very nature of turning a string into an integer is more complicated by necessity. Any valid 16 bit number can be turned into a string from bases 2 to 36 by the “intstr” function. However, strings can contain characters that are invalid to be converted as numbers. There is also the issue that both capital and lowercase letters might be used in radices higher than ten. In the program

above, I used a binary integer string for an example, but in real applications, such as a famous program I wrote called “chastehex”, it is necessary to write a function that can gracefully handle not only the decimal digits ‘0’ to ‘9’ but also handle letters ‘A’ to ‘Z’ or ‘a’ to ‘z’.

Below is the full source code of the strint function as written for 16 bit DOS Assembly programs. I spent more time writing this function than the three previous functions combined, but it was necessary for the programs I intended to write! Comments are included although more explanation may be required for some to understand it.

The Function

```

1  ;this function converts a string pointed to by ax into an integer returned in
   ↳ ax instead
2  ;it is a little complicated because it has to account for whether the character
   ↳ in
3  ;a string is a decimal digit 0 to 9, or an alphabet character for bases higher
   ↳ than ten
4  ;it also checks for both uppercase and lowercase letters for bases 11 to 36
5  ;finally, it checks if that letter makes sense for the base.
6  ;For example, G to Z cannot be used in hexadecimal, only A to F can
7  ;The purpose of writing this function was to be able to accept user input as
   ↳ integers
8
9  strint:
10
11  mov bx,ax ;copy string address from ax to bx because ax will be replaced soon!
12  mov ax,0
13
14  read_strint:
15  mov cx,0 ; zero cx so only lower 8 bits are used
16  mov cl,[bx] ;copy byte/character at address bx to cl register (lowest part of
   ↳ cx)
17  inc bx ;increment bx to be ready for next character
18  cmp cl,0 ; compare this byte with 0
19  jz strint_end ; if comparison was zero, this is the end of string
20
21  ;if char is below '0' or above '9', it is outside the range of these and is not
   ↳ a digit
22  cmp cl,'0'
23  jb not_digit
24  cmp cl,'9'
25  ja not_digit
26
27  ;but if it is a digit, then correct and process the character

```

```
28  is_digit:
29  sub cl,'0'
30  jmp process_char
31
32  not_digit:
33  ;it isn't a decimal digit, but it could be perhaps an alphabet character
34  ;which could be a digit in a higher base like hexadecimal
35  ;we will check for that possibility next
36
37  ;if char is below 'A' or above 'Z', it is outside the range of these and is not
   ↪ capital letter
38  cmp cl,'A'
39  jb not_upper
40  cmp cl,'Z'
41  ja not_upper
42
43  is_upper:
44  sub cl,'A'
45  add cl,10
46  jmp process_char
47
48  not_upper:
49
50  ;if char is below 'a' or above 'z', it is outside the range of these and is not
   ↪ lowercase letter
51  cmp cl,'a'
52  jb not_lower
53  cmp cl,'z'
54  ja not_lower
55
56  is_lower:
57  sub cl,'a'
58  add cl,10
59  jmp process_char
60
61  not_lower:
62
63  ;if we have reached this point, result invalid and end function
64  jmp strint_end
65
66  process_char:
67
68  cmp cx,[radix] ;compare char with radix
69  jnb strint_end ;if this value is above or equal to radix, it is too high
   ↪ despite being a valid digit/alpha
70
71  mov dx,0 ;zero dx because it is used in mul sometimes
72  mul word [radix] ;mul ax with radix
73  add ax,cx
74
```

```
75 jmp read_strint ;jump back and continue the loop if nothing has exited it
76
77 strint_end:
78
79 ret
```

If you run the program at the top of this chapter (remember the source is available on github but also can be pieced together from this book alone), it will produce this output.

The Output

```
1 This is the year I was born
2 binary: 11111000011
3 hexadecimal: 7C3
4 decimal: 1987
```

These three forms of displaying the same number are quite obviously the most useful radices that programmers must learn.

Binary is what computers understand. Without knowing that everything is bits of only 0 or 1, very little about computers makes sense without a complete understanding of the Binary Numeral System.

Hexadecimal is the short form of Binary because every four bits equals one digit in Hexadecimal. For this reason, hex editors for editing binary files are much more common than binary editors. It just takes less typing and disk space.

Decimal actually has no value other than what humans have placed on it. Base ten is not special, and computers don't understand it as well as Binary, but humans expect numbers to be in this form, and by extension, all the video games also display numbers in this form.

If I told people that I was born in 11111000011, they would think I am really old and should be dead by now. If I tell them I was born in 1987, they will know that I am 38 years old at the time I am writing this book. However, both numbers mean the same thing using different number bases.

The reason I mention this is that I want you to research different bases/radixes of number systems because it will make you a better programmer. This knowledge will also help you understand the program I share in chapter 10 of this book.

Chapter 7: Translating Assembly to Other Programming Languages

This chapter is going to be a weird one, because most people don't start with assembly language before moving to higher level languages. In fact most people would probably recommend against Assembly as a first programming language.

But for the purpose of this chapter alone, I will be assuming that you have been following the first 6 chapters of this Assembly book and want to know how this knowledge can be used to translate the Assembly into other languages like C and C++. This is actually very easy to do because the other languages are easier and have built in functions for you to use.

So what I did is write a test suite program. It makes use of the core 4 of my chastelib functions (putstring,putint,intstr,rint) as well as some other utility functions just for displaying single characters, lines, and spaces.

In this chapter, I will be including the entire source code of the main program "main.asm" as well as "chastelib16.asm" which is the included file containing all the useful output functions. Snippets of these have been included throughout the book but by including them all in this chapter, you can be sure that you have the most updated and commented version of the source code. This will become more important later when I show you the C equivalent program.

main.asm

```

1  org 100h
2
3  main:
4
5  mov ax,main_string
6  call putstring
7
8  mov word[radix],16      ;can choose radix for integer output!
9  mov word[int_width],1
10 mov byte[int_newline],0
11
12 mov ax,input_string_int ;address of input string to convert to integer using
   ↳ current radix
13 call strint             ;call strint to return the string in ax register
14 mov bx,ax               ;bx=ax (copy the converted value returned in ax to bx)
15
16 mov ax,0
17 loop1:
18
19 mov word[radix],2        ;set radix to binary
20 mov word[int_width],8    ;width of 8 bits
21 call putint
22 call putspace
23 mov word[radix],16       ;set radix to hexadecimal
24 mov word[int_width],2    ;width of 2 hex digits
25 call putint
26 call putspace
27 mov word[radix],10       ;set radix to decimal (what humans read)
28 mov word[int_width],3    ;width of 3 decimal digits
29 call putint
30
31 cmp al,0x20 ;check if al is in printable range
32 jb not_char ;if not then jump to not_char label
33 cmp al,0x7E
34 ja not_char
35
36 call putspace
37 call putchar ;print the character if it is in the range 0x20 to 0x7E
38
39 not_char:    ;jump here if character is outside range to print
40
41 call putline ;print newline before the next loop
42
43 inc ax
44 cmp ax,bx;
45 jnz loop1
46
47 mov ax,4C00h ;DOS system call number ah=0x4C to exit program with ah=0x00 as
   ↳ return value
48 int 21h      ;DOS interrupt to exit the program with numbers on previous line

```



```

49
50 ;A string to test if output works
51 main_string db 'Official test suite for the DOS Assembly version of
    ↳ chastelib.',0Ah,0
52
53 ;test string of integer for input
54 input_string_int db '100',0
55
56 include 'chastelib16.asm' ; use %include if assembling with NASM instead of
    ↳ FASM.
57
58 ; This 16 bit DOS Assembly source has been formatted for the FASM assembler.
59 ; In order to run it, you will need the DOSBox emulator or something similar.
60 ; First, assemble it into a binary file. FASM will automatically add
61 ; the .com extension because of the "org 100h" command.
62 ;
63 ; fasm main.asm
64 ;
65 ; Then you will need to open DOSBOX and mount the folder that it is in.
66 ; For example:
67 ;
68 ; mount c ~/.dos
69 ; c:
70 ;
71 ; Then, you will be able to just run the main.com.
72 ;
73 ; main

```

chastelib16.asm

```

1 ; This file is where I keep my function definitions.
2 ; These are usually my string and integer output routines.
3
4 ;this is my best putstring function for DOS because it uses call 40h of
    ↳ interrupt 21h
5 ;this means that it works in a similar way to my Linux Assembly code
6 ;the plan is to make both my DOS and Linux functions identical except for the
    ↳ size of registers involved
7
8 stdout dw 1 ; variable for standard output so that it can theoretically be
    ↳ redirected
9
10 putstring:
11
12 push ax
13 push bx
14 push cx

```

```

15  push dx
16
17  mov bx,ax                ;copy ax to bx for use as index register
18
19  putstring_strlen_start:  ;this loop finds the length of the string as part of
    ↪ the putstring function
20
21  cmp byte[bx],0           ;compare this byte with 0
22  jz putstring_strlen_end  ;if comparison was zero, jump to loop end because we
    ↪ have found the length
23  inc bx                  ;increment bx (add 1)
24  jmp putstring_strlen_start ;jump to the start of the loop and keep trying until
    ↪ we find a zero
25
26  putstring_strlen_end:
27
28  sub bx,ax                ; sub ax from bx to get the difference for number of
    ↪ bytes
29  mov cx,bx                ; mov bx to cx
30  mov dx,ax                ; dx will have address of string to write
31
32  mov ah,40h               ; select DOS function 40h write
33  mov bx,[stdout]          ; file handle 1=stdout
34  int 21h                  ; call the DOS kernel
35
36  pop dx
37  pop cx
38  pop bx
39  pop ax
40
41  ret
42
43
44
45  ;this is the location in memory where digits are written to by the intstr
    ↪ function
46
47  int_string db 16 dup '?' ;enough bytes to hold maximum size 16-bit binary
    ↪ integer
48
49  ;this is the end of the integer string optional line feed and terminating zero
50  ;clever use of this label can change the ending to be a different character
    ↪ when needed
51
52  int_newline db 0Dh,0Ah,0 ;the proper way to end a line in DOS/Windows
53
54  radix dw 2 ;radix or base for integer output. 2=binary, 8=octal, 10=decimal,
    ↪ 16=hexadecimal
55  int_width dw 8
56

```

```
57  intstr:
58
59  mov bx,int_newline-1 ;find address of lowest digit(just before the newline 0Ah)
60  mov cx,1
61
62  digits_start:
63
64  mov dx,0;
65  div word [radix]
66  cmp dx,10
67  jb decimal_digit
68  jnb hexadecimal_digit
69
70  decimal_digit: ;we go here if it is only a digit 0 to 9
71  add dx,'0'
72  jmp save_digit
73
74  hexadecimal_digit:
75  sub dx,10
76  add dx,'A'
77
78  save_digit:
79
80  mov [bx],dl
81  cmp ax,0
82  jz intstr_end
83  dec bx
84  inc cx
85  jmp digits_start
86
87  intstr_end:
88
89  prefix_zeros:
90  cmp cx,[int_width]
91  jnb end_zeros
92  dec bx
93  mov byte[bx], '0'
94  inc cx
95  jmp prefix_zeros
96  end_zeros:
97
98  mov ax,bx ; store string in ax for display later
99
100 ret
101
102
103
104 ;function to print string form of whatever integer is in ax
105 ;The radix determines which number base the string form takes.
106 ;Anything from 2 to 36 is a valid radix
```

```
107 ;in practice though, only bases 2,8,10,and 16 will make sense to other
    ↳ programmers
108 ;this function does not process anything by itself but calls the combination of
    ↳ my other
109 ;functions in the order I intended them to be used.
110
111 putint:
112
113 push ax
114 push bx
115 push cx
116 push dx
117
118 call intstr
119 call putstring
120
121 pop dx
122 pop cx
123 pop bx
124 pop ax
125
126 ret
127
128
129
130
131
132
133
134
135 ;this function converts a string pointed to by ax into an integer returned in
    ↳ ax instead
136 ;it is a little complicated because it has to account for whether the character
    ↳ in
137 ;a string is a decimal digit 0 to 9, or an alphabet character for bases higher
    ↳ than ten
138 ;it also checks for both uppercase and lowercase letters for bases 11 to 36
139 ;finally, it checks if that letter makes sense for the base.
140 ;For example, G to Z cannot be used in hexadecimal, only A to F can
141 ;The purpose of writing this function was to be able to accept user input as
    ↳ integers
142
143 strint:
144
145 mov bx,ax ;copy string address from ax to bx because ax will be replaced soon!
146 mov ax,0
147
148 read_strint:
149 mov cx,0 ; zero cx so only lower 8 bits are used
```

```
150 mov cl,[bx] ;copy byte/character at address bx to cl register (lowest part of
    ↪ cx)
151 inc bx ;increment bx to be ready for next character
152 cmp cl,0 ; compare this byte with 0
153 jz strint_end ; if comparison was zero, this is the end of string
154
155 ;if char is below '0' or above '9', it is outside the range of these and is not
    ↪ a digit
156 cmp cl,'0'
157 jb not_digit
158 cmp cl,'9'
159 ja not_digit
160
161 ;but if it is a digit, then correct and process the character
162 is_digit:
163 sub cl,'0'
164 jmp process_char
165
166 not_digit:
167 ;it isn't a decimal digit, but it could be perhaps an alphabet character
168 ;which could be a digit in a higher base like hexadecimal
169 ;we will check for that possibility next
170
171 ;if char is below 'A' or above 'Z', it is outside the range of these and is not
    ↪ capital letter
172 cmp cl,'A'
173 jb not_upper
174 cmp cl,'Z'
175 ja not_upper
176
177 is_upper:
178 sub cl,'A'
179 add cl,10
180 jmp process_char
181
182 not_upper:
183
184 ;if char is below 'a' or above 'z', it is outside the range of these and is not
    ↪ lowercase letter
185 cmp cl,'a'
186 jb not_lower
187 cmp cl,'z'
188 ja not_lower
189
190 is_lower:
191 sub cl,'a'
192 add cl,10
193 jmp process_char
194
195 not_lower:
```

```
196
197 ;if we have reached this point, result invalid and end function
198 jmp strint_end
199
200 process_char:
201
202 cmp cx,[radix] ;compare char with radix
203 jnb strint_end ;if this value is above or equal to radix, it is too high
    ↳ despite being a valid digit/alpha
204
205 mov dx,0 ;zero dx because it is used in mul sometimes
206 mul word [radix] ;mul ax with radix
207 add ax,cx
208
209 jmp read_strint ;jump back and continue the loop if nothing has exited it
210
211 strint_end:
212
213 ret
214
215
216
217 ;returns in al register a character from the keyboard
218 getchr:
219
220 mov ah,1
221 int 21h
222
223 ret
224
225 ;the next utility functions simply print a space or a newline
226 ;these help me save code when printing lots of things for debugging
227
228 space db ' ',0
229 line db 0Dh,0Ah,0
230
231 putspace:
232 push ax
233 mov ax,space
234 call putstring
235 pop ax
236 ret
237
238 putline:
239 push ax
240 mov ax,line
241 call putstring
242 pop ax
243 ret
244
```

```
245 ;a function for printing a single character that is the value of al
246
247 char: db 0,0
248
249 putchar:
250 push ax
251 mov [char],al
252 mov ax,char
253 call putstring
254 pop ax
255 ret
```

Now that you have the full source code, you can either copy and paste it from the PDF or epub edition (if you purchased the Leanpub edition) or you can download it directly from the github repository I have linked to at least twice in this book already.

But you don't even have to assemble and run it to see what it does because I am going to show you the entire output that it generates!

Assembly Test Suite Output

```
1 This program is the official test suite for the DOS Assembly version of
  ↳ chastelib.
2 00000000 00 000
3 00000001 01 001
4 00000010 02 002
5 00000011 03 003
6 00000100 04 004
7 00000101 05 005
8 00000110 06 006
9 00000111 07 007
10 00001000 08 008
11 00001001 09 009
12 00001010 0A 010
13 00001011 0B 011
14 00001100 0C 012
15 00001101 0D 013
16 00001110 0E 014
17 00001111 0F 015
18 00010000 10 016
19 00010001 11 017
20 00010010 12 018
21 00010011 13 019
22 00010100 14 020
23 00010101 15 021
```

```
24 00010110 16 022
25 00010111 17 023
26 00011000 18 024
27 00011001 19 025
28 00011010 1A 026
29 00011011 1B 027
30 00011100 1C 028
31 00011101 1D 029
32 00011110 1E 030
33 00011111 1F 031
34 00100000 20 032
35 00100001 21 033 !
36 00100010 22 034 "
37 00100011 23 035 #
38 00100100 24 036 $
39 00100101 25 037 %
40 00100110 26 038 &
41 00100111 27 039 '
42 00101000 28 040 (
43 00101001 29 041 )
44 00101010 2A 042 *
45 00101011 2B 043 +
46 00101100 2C 044 ,
47 00101101 2D 045 -
48 00101110 2E 046 .
49 00101111 2F 047 /
50 00110000 30 048 0
51 00110001 31 049 1
52 00110010 32 050 2
53 00110011 33 051 3
54 00110100 34 052 4
55 00110101 35 053 5
56 00110110 36 054 6
57 00110111 37 055 7
58 00111000 38 056 8
59 00111001 39 057 9
60 00111010 3A 058 :
61 00111011 3B 059 ;
62 00111100 3C 060 <
63 00111101 3D 061 =
64 00111110 3E 062 >
65 00111111 3F 063 ?
66 01000000 40 064 @
67 01000001 41 065 A
68 01000010 42 066 B
69 01000011 43 067 C
70 01000100 44 068 D
71 01000101 45 069 E
72 01000110 46 070 F
73 01000111 47 071 G
```



```
74 01001000 48 072 H
75 01001001 49 073 I
76 01001010 4A 074 J
77 01001011 4B 075 K
78 01001100 4C 076 L
79 01001101 4D 077 M
80 01001110 4E 078 N
81 01001111 4F 079 O
82 01010000 50 080 P
83 01010001 51 081 Q
84 01010010 52 082 R
85 01010011 53 083 S
86 01010100 54 084 T
87 01010101 55 085 U
88 01010110 56 086 V
89 01010111 57 087 W
90 01011000 58 088 X
91 01011001 59 089 Y
92 01011010 5A 090 Z
93 01011011 5B 091 [
94 01011100 5C 092 \
95 01011101 5D 093 ]
96 01011110 5E 094 ^
97 01011111 5F 095 _
98 01100000 60 096 `
99 01100001 61 097 a
100 01100010 62 098 b
101 01100011 63 099 c
102 01100100 64 100 d
103 01100101 65 101 e
104 01100110 66 102 f
105 01100111 67 103 g
106 01101000 68 104 h
107 01101001 69 105 i
108 01101010 6A 106 j
109 01101011 6B 107 k
110 01101100 6C 108 l
111 01101101 6D 109 m
112 01101110 6E 110 n
113 01101111 6F 111 o
114 01110000 70 112 p
115 01110001 71 113 q
116 01110010 72 114 r
117 01110011 73 115 s
118 01110100 74 116 t
119 01110101 75 117 u
120 01110110 76 118 v
121 01110111 77 119 w
122 01111000 78 120 x
123 01111001 79 121 y
```

```
124 01111010 7A 122 z
125 01111011 7B 123 {
126 01111100 7C 124 |
127 01111101 7D 125 }
128 01111110 7E 126 ~
129 01111111 7F 127
130 10000000 80 128
131 10000001 81 129
132 10000010 82 130
133 10000011 83 131
134 10000100 84 132
135 10000101 85 133
136 10000110 86 134
137 10000111 87 135
138 10001000 88 136
139 10001001 89 137
140 10001010 8A 138
141 10001011 8B 139
142 10001100 8C 140
143 10001101 8D 141
144 10001110 8E 142
145 10001111 8F 143
146 10010000 90 144
147 10010001 91 145
148 10010010 92 146
149 10010011 93 147
150 10010100 94 148
151 10010101 95 149
152 10010110 96 150
153 10010111 97 151
154 10011000 98 152
155 10011001 99 153
156 10011010 9A 154
157 10011011 9B 155
158 10011100 9C 156
159 10011101 9D 157
160 10011110 9E 158
161 10011111 9F 159
162 10100000 A0 160
163 10100001 A1 161
164 10100010 A2 162
165 10100011 A3 163
166 10100100 A4 164
167 10100101 A5 165
168 10100110 A6 166
169 10100111 A7 167
170 10101000 A8 168
171 10101001 A9 169
172 10101010 AA 170
173 10101011 AB 171
```

```
174 10101100 AC 172
175 10101101 AD 173
176 10101110 AE 174
177 10101111 AF 175
178 10110000 B0 176
179 10110001 B1 177
180 10110010 B2 178
181 10110011 B3 179
182 10110100 B4 180
183 10110101 B5 181
184 10110110 B6 182
185 10110111 B7 183
186 10111000 B8 184
187 10111001 B9 185
188 10111010 BA 186
189 10111011 BB 187
190 10111100 BC 188
191 10111101 BD 189
192 10111110 BE 190
193 10111111 BF 191
194 11000000 C0 192
195 11000001 C1 193
196 11000010 C2 194
197 11000011 C3 195
198 11000100 C4 196
199 11000101 C5 197
200 11000110 C6 198
201 11000111 C7 199
202 11001000 C8 200
203 11001001 C9 201
204 11001010 CA 202
205 11001011 CB 203
206 11001100 CC 204
207 11001101 CD 205
208 11001110 CE 206
209 11001111 CF 207
210 11010000 D0 208
211 11010001 D1 209
212 11010010 D2 210
213 11010011 D3 211
214 11010100 D4 212
215 11010101 D5 213
216 11010110 D6 214
217 11010111 D7 215
218 11011000 D8 216
219 11011001 D9 217
220 11011010 DA 218
221 11011011 DB 219
222 11011100 DC 220
223 11011101 DD 221
```

```
224 11011110 DE 222
225 11011111 DF 223
226 11100000 E0 224
227 11100001 E1 225
228 11100010 E2 226
229 11100011 E3 227
230 11100100 E4 228
231 11100101 E5 229
232 11100110 E6 230
233 11100111 E7 231
234 11101000 E8 232
235 11101001 E9 233
236 11101010 EA 234
237 11101011 EB 235
238 11101100 EC 236
239 11101101 ED 237
240 11101110 EE 238
241 11101111 EF 239
242 11110000 F0 240
243 11110001 F1 241
244 11110010 F2 242
245 11110011 F3 243
246 11110100 F4 244
247 11110101 F5 245
248 11110110 F6 246
249 11110111 F7 247
250 11111000 F8 248
251 11111001 F9 249
252 11111010 FA 250
253 11111011 FB 251
254 11111100 FC 252
255 11111101 FD 253
256 11111110 FE 254
257 11111111 FF 255
```

main.c (The C Test Suite)

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include "chastelib.h"
4
5  int main(int argc, char *argv[])
6  {
7      int a=0,b;
8
9      radix=16;
10     int_width=1;
11
12    _putstr("Official test suite for the C version of chastelib.\n");
13
14     b=strint("100"); /*convert string to an integer*/
15     if(strint_errors) /*if there are errors, print some messages and exit the
16     ↪ program*/
17     {
18        _putstr("Input string passed to strint function contains errors.\n");
19        _putstr("Please fix the invalid characters/radix and try again.\n");
20         return 0;
21     }
22     while(a<b)
23     {
24         radix=2;
25         int_width=8;
26         putint(a);
27        _putstr(" ");
28         radix=16;
29         int_width=2;
30         putint(a);
31        _putstr(" ");
32         radix=10;
33         int_width=3;
34         putint(a);
35
36         if(a>=0x20 && a<=0x7E)
37         {
38            _putstr(" ");
39             putchar(a);
40         }
41
42        _putstr("\n");
43         a+=1;
44     }
45
46     return 0;
47 }
```

The C version above does the exact same steps as the assembly version,

but is calling functions that do very much the same steps as the assembly version. I will show you the contents of the included file “chastelib.h” next. Be prepared for a slightly less painful mess of code! However, much like the Assembly version it is heavily commented to help with understanding it.

chastelib.h (The C chastelib library)

```

1  /*
2  This file is a C library of functions written by Chastity White Rose. The
   ↳ functions are for converting strings into integers and integers into
   ↳ strings.
3  I did it partly for future programming plans and also because it helped me
   ↳ learn a lot in the process about how pointers work
4  as well as which features the standard library provides, and which things I
   ↳ need to write my own functions for.
5
6  As it turns out, the integer output routines for C are too limited for my
   ↳ tastes. This library corrects this problem.
7  Using the global variables and functions in this file, integers can be output
   ↳ in bases/radixes 2 to 36.
8
9  Although this code is commented, I have also written a readme.md file designed
   ↳ to explain the usage of these functions and the philosophy behind them.
10 */
11
12 /*
13 These following lines define a static array with a size big enough to store
   ↳ the digits of an integer, including padding it with extra zeroes.
14 The integer conversion function (intstr) always references a pointer to this
   ↳ global string, and this allows other C standard library functions
15 such as printf to display the integers to standard output or even possibly to
   ↳ files.
16 This string can be repurposed for absolutely anything I desire.
17 */
18
19
20 #define usl 0x100 /*usl stands for Unsigned or Universal String Length.*/
21 char int_string[usl+1]; /*global string which will be used to store string of
   ↳ integers. Size is usl+1 for terminating zero*/
22
23 /*radix or base for integer output. 2=binary, 8=octal, 10=decimal,
   ↳ 16=hexadecimal*/
24 int radix=2;
25 /*default minimum digits for printing integers*/
26 int int_width=1;
27
28 /*

```

```

29 The intstr function is one that I wrote because the standard library can
   ↳ display integers as decimal, octal, or hexadecimal, but not any other
   ↳ bases(including binary, which is my favorite).
30
31 My function corrects this, and in my opinion, such a function should have been
   ↳ part of the standard library, but I'm not complaining because now I have my
   ↳ own, which I can use forever!
32 More importantly, it can be adapted for any programming language in the world
   ↳ if I learn the basics of that language. That being said, C is the best
   ↳ language and I will use it forever.
33 */
34
35 char *intstr(unsigned int i)    /*Chastity's supreme integer to string
   ↳ conversion function*/
36 {
37     int width=0;                /*the width or how many digits including
   ↳ prefixed zeros are printed*/
38     char *s=int_string+usl;     /*a pointer starting to the place where we will
   ↳ end the string with zero*/
39     *s=0;                       /*set the zero that terminates the string in the
   ↳ C language*/
40     while(i!=0 || width<int_width) /*loop to fill the string with every required
   ↳ digit plus prefixed zeros*/
41     {
42         s--;                    /*decrement the pointer to go left for current
   ↳ digit placing*/
43         *s=i%radix;             /*get the remainder of division by the radix or
   ↳ base*/
44         i/=radix;               /*divide the input by radix*/
45         if(*s<10){*s+='0';}     /*convert digits 0 to 9 to the ASCII character
   ↳ for that digit*/
46         else{*s=*s+'A'-10;}     /*for digits higher than 9, convert to letters
   ↳ starting at A*/
47         width++;                /*increment the width so we know when enough
   ↳ digits are saved*/
48     }
49     return s;                  /*return this string to be used by
   ↳ putstr,printf,std::cout or whatever*/
50 }
51
52 /*
53 The strint_errors variable is used to keep track of how many errors happened in
   ↳ the strint function.
54 The following errors can occur:
55
56 Radix is not in range 2 to 36
57 Character is not a number 0 to 9 or alphabet A to Z (in either case)
58 Character is alphanumeric but is not valid for current radix
59

```

```

60 If any of these errors happen, error messages are printed to let the programmer
   ↳ or user know what went wrong in the string that was passed to the function.
61 If getting input from the keyboard, the strint_errors variable can be used in a
   ↳ conditional statement to tell them to try again and recall the code that
   ↳ grabs user input.
62 */
63
64 int strint_errors = 0;
65
66 /*
67 The strint function is my own replacement for the strtol function from the C
   ↳ standard library.
68 I didn't technically need to make this function because the functions from
   ↳ stdlib.h can already convert strings from bases 2 to 36 into integers.
69 However, my function is simpler because it only requires 2 arguments instead
   ↳ of three, and it also does not handle negative numbers.
70 I have never needed negative integers, but if I ever do, I can use the standard
   ↳ functions or write my own in the future.
71 */
72
73 int strint(const char *s)
74 {
75     int i=0;
76     char c;
77     strint_errors = 0; /*set zero errors before we parse the string*/
78     if( radix<2 || radix>36 ){ strint_errors++; printf("Error: radix %i is out of
   ↳ range!\n",radix);}
79     while( *s == ' ' || *s == '\n' || *s == '\t' ){s++;} /*skip whitespace at
   ↳ beginning*/
80     while(*s!=0)
81     {
82         c=*s;
83         if( c >= '0' && c <= '9' ){c-='0';}
84         else if( c >= 'A' && c <= 'Z' ){c-='A';c+=10;}
85         else if( c >= 'a' && c <= 'z' ){c-='a';c+=10;}
86         else if( c == ' ' || c == '\n' || c == '\t' ){break;}
87         else{ strint_errors++; printf("Error: %c is not an alphanumeric
   ↳ character!\n",c);break;}
88         if(c>=radix){ strint_errors++; printf("Error: %c is not a valid character for
   ↳ radix %i\n",*s,radix);break;}
89         i*=radix;
90         i+=c;
91         s++;
92     }
93     return i;
94 }
95
96 /*
97 This function prints a string using fwrite.

```



```

98  This algorithm is the best C representation of how my Assembly programs also
    ↪ work.
99  Its true purpose is to be used in the putint function for conveniently
    ↪ printing integers,
100 but it can print any valid string.
101 */
102
103 int putstring(const char *s)
104 {
105     int count=0;           /*used to calculate how many bytes will be written*/
106     const char *p=s;       /*pointer used to find terminating zero of string*/
107     while(*p){p++;}        /*loop until zero found and immediately exit*/
108     count=p-s;             /*count is the difference of pointers p and s*/
109     fwrite(s,1,count,stdout); /*https://cppreference.com/w/c/io/fwrite.html*/
110     return count;          /*return how many bytes were written*/
111 }
112
113 /*
114  A function pointer named putstr which is a shorter name for calling putstring
115  But this doesn't exist just to save bytes of source files. Otherwise I
    ↪ wouldn't have these huge comments!
116  This exists so that all strings can be redirected to another function for
    ↪ output.
117  For example, if the strings were written to a log file during a game which
    ↪ didn't use a terminal.
118
119  But the most common use case is "putstr=addstr" when using the ncurses library
    ↪ to manage
120  terminal control functions for a text based game. Having the putstr pointer
    ↪ allows me to
121  include this same source file and use it for ncurses based projects.
122 */
123 int (*putstr)(const char *)=putstring;
124
125 /*
126  This function uses both intstr and putstring to print an integer in the
    ↪ currently selected radix and width.
127 */
128
129 void putint(unsigned int i)
130 {
131     putstr(intstr(i));
132 }
133
134 /*
135  Those four functions above are the core of chastelib.
136  While there may be extensions written for specific programs, these functions
    ↪ are essential for absolutely every program I write.
137

```

```

138 The only reason you would not need them is if you only output numbers in
    ↳ decimal or hexadecimal, because printf in C can do all that just fine.
139 However, the reason my core functions are superior to printf is that printf
    ↳ and its family of functions require the user to memorize all the arcane
    ↳ symbols for format specifiers.
140
141 The core functions are primarily concerned with standard output and the
    ↳ conversion of strings and integers. They do not deal with input from the
    ↳ keyboard or files. A separate extension will be written for my programs
    ↳ that need these features.
142 */

```

Now that you have witnessed the largest dump of code to ever be included in a chapter of a book, I want you to look it over carefully and you will notice that there is almost direct equivalence between the Assembly version and the C version.

Here is a detailed breakdown of how both versions operate despite the language syntax looking completely different.

- **putstring** finds the terminating zero to calculate string length and prints that length of bytes. It achieves this by using the `fwrite` function which is part of the standard library. Just like the “`ah=40h`” DOS call, it must be given the arguments to say “Start at this address and write exactly this number of bytes to standard output!”. It also uses tons of pointer arithmetic in both the C and assembly versions. In this case, I would argue that the Assembly version might be easier to read than the C version. C lets a person create some weird looking code with the syntax used for pointers
- **intstr** converts an integer into a string at a specific predetermined address and then returns a pointer to this address from the function. In both the C and Assembly version, the process of repeated division by the radix (also known as number base) while the integer is above zero or the string has not reached the minimum width or length I want the string to have. It will prefix the string with extra zeros just so it lines up perfectly as you saw in the output earlier in this chapter.
- **putint** is merely a convenience function the calls **intstr** and then **putstr** to convert and print an integer in one step. Functions are designed to repeat frequent operations to reduce code size and save programmer time. Though to be honest, if your time was valuable to you, you probably wouldn’t be reading a DOS assembly language book. Thanks for reading my book anyway!

- **strint** does the opposite of **intstr** as you might guess from its name. It converts a string into an integer. Its usefulness is not fully obvious here because the example program reads a predefined string. Ordinarily, you would get user input from either the keyboard during the program or from command line arguments passed to the program before it starts. One small piece of advice though, if you want to accept user input, C is a better language than Assembly because I haven't been successful in getting anyone to assembly and run my DOS assembly programs anyway!

You probably noticed other functions like `putchar`, `putline`, and `putspace`. I made these convenience functions in assembly because there are times when you need to print a character to separate numbers. Usually spaces and newlines are the most important. The `putchar` function exists in the C standard library since at least 1989 and probably much earlier.

Portable Assembly Language

C has been called a portable assembly language because C compilers translate C code into assembly language and then link it with the precompiled functions in other libraries. In short, they do the opposite process of what I have done in this chapter. I wrote these string and integer output routines because they didn't exist in assembly language by default.

C already has `printf`, `putchar`, and `fwrite`. These are more than enough to handle outputting text including numbers without having to use the functions I have written. But in any case, I provided them in this chapter for helping people understand how these operations are done.

But when you are trying to write programs for DOS, assembly is still better because there are not enough easy to find C compilers that still work in 16 bit DOS mode. There was one made by the company Borland known as Turbo C. If you are lucky enough to find it on the internet and get it running, you might enjoy it.

C++ is a programming language that came after C and includes everything C has plus more. However, this book is about assembly and this chapter was but a brief introduction to the idea of translating assembly language into C for the purpose of having something portable to all platforms.

My other book, [Chastity's Code Cookbook](#) uses mostly C code as an introduction to programming. If you liked this chapter, consider reading it for more nerdy programming content.

As far as DOS goes, Assembly is still the preferred language due to the fact that it allows us to create very small programs as you have seen throughout this book.

And by small, I mean the size of them when assembled! The source code is usually much larger than the assembled binary machine code!

Chapter 8: Going from DOS to Linux or Windows

In the unlikely event that you have read the first 7 chapters of this book, I am going to assume you are a pretty hard core computer user. What I can say for sure is that you are the type of person who reads books or blog posts about technical details. DOS is an operating system that tends to only be used by nerds who love reading text and efficient operations at the command line.

Sadly to say, our kind is dying out. At the time of writing this I am 38 years old and there are few people who remember the old way computers were used. DOS is mostly seen as a dead platform and it is not usually used except by programmers and hard core gamers who still run their favorite games in a DOS emulator, although I cannot fail to mention that FreeDOS is available as a real DOS system.

<https://www.freedos.org/>

But most people know nothing about DOS because the popular operating systems available today are Windows, MacOS and Linux.

If you have enjoyed programming in Assembly, I do have some helpful tips on how you can apply most of the same information to start Assembly in Linux.

As far as Windows or MacOS go, I cannot help you much with that because I don't use proprietary operating systems if I have a choice. These operating systems don't allow you to simply load registers and call interrupts to print things on the screen.

Linux, however, works very much like DOS does. If you know how to load the registers correctly and use a system call, you can print strings of text just like in DOS except MUCH faster because you will be running natively instead of in an emulator as in the DOS examples from the rest of this book.

I cannot cover the details of installing a Linux operating system because there are many choices. However I recommend Debian because it has been my main distro for years. Therefore, the following two programs that I will show

you in this chapter have both been tested to work on my 64 bit Intel PC running Debian 12 (bookworm).

Remember, although DOS was a 16 bit system, modern Linux processors and distros usually support 32 or 64 bit code. Therefore, I will be showing you a small program using the FASM assembler that prints text using a Linux version of the putstring function. It behaves the same as the DOS version behaves in chapter 2.

main.asm (32 bit)

```
1  format ELF executable
2  entry main
3
4  main:
5
6  mov eax,main_string
7  call putstring
8
9  mov eax, 1 ; invoke SYS_EXIT (kernel opcode 1)
10 mov ebx, 0 ; return 0 status on exit - 'No Errors'
11 int 80h
12
13 ;A string to test if output works
14 main_string db 'This program runs in Linux!',0Ah,0
15
16 putstring:
17
18 push eax
19 push ebx
20 push ecx
21 push edx
22
23 mov ebx,eax ; copy eax to ebx. ebx will be used as index to the string
24
25 putstring_strlen_start: ; this loop finds the length of the string as part of
   ↳ the putstring function
26
27 cmp [ebx],byte 0 ; compare byte at address ebx with 0
28 jz putstring_strlen_end ; if comparison was zero, jump to loop end because we
   ↳ have found the length
29 inc ebx
30 jmp putstring_strlen_start
31
32 putstring_strlen_end:
33 sub ebx,eax ;By subtracting the start of the string with the current address,
   ↳ we have the length of the string.
```

```

34
35 ; Write string using Linux Write system call. Reference for 32 bit x86 syscalls
   ↳ is below.
36 ; https://www.chromium.org/chromium-os/developer-library/reference/linux-cons
   ↳ tants/syscalls/#x86-32-bit
37
38 mov edx,ebx      ;number of bytes to write
39 mov ecx,eax      ;pointer/address of string to write
40 mov ebx,1        ;write to the STDOUT file
41 mov eax,4        ;invoke SYS_WRITE (kernel opcode 4 on 32 bit systems)
42 int 80h          ;system call to write the message
43
44 pop edx
45 pop ecx
46 pop ebx
47 pop eax
48
49 ret ; this is the end of the putstring function return to calling location
50
51 ; This Assembly source file has been formatted for the FASM assembler.
52 ; The following 3 commands assemble, give executable permissions, and run the
   ↳ program
53 ;
54 ; fasm main.asm
55 ; chmod +x main
56 ; ./main

```

The program above uses only two system calls. One is the call to exit the program. The other is the write call which is the same as the DOS function 0x40 of interrupt 0x21; However, the usage of the registers is not in the same order. However, these registers: eax,ebx,ecx,edx are the same registers except that they are extended to 32 bits. That is why they have an e in their name.

But if you take the time to study it, you will see that it does the exact same process of finding the length of the string by the terminating zero and then loading the registers in such a way that the operating system knows what function we are calling, which handle we are writing to, how many bytes to write, and where the data is in memory which will be written.

Next I will show you the 64-bit equivalent that works the same way but uses different numbers for the system calls.

main.asm 64 bit

```

1  format ELF64 executable
2  entry main
3
4  main: ; the main function of our assembly function, just as if I were writing C.
5
6  mov rax,main_string ; move the address of main_string into rax register
7  call putstring
8
9  mov rax, 60 ; invoke SYS_EXIT (kernel opcode 60 on 64 bit systems)
10 mov rdi,0    ; return 0 status on exit - 'No Errors'
11 syscall
12
13 ;A string to test if output works
14 main_string db 'This program runs in Linux!','0Ah,0
15
16 putstring:
17
18 push rax
19 push rbx
20 push rcx
21 push rdx
22
23 mov rbx,rax ; copy rax to rbx as well. Now both registers have the address of
   ↪ the main_string
24
25 putstring_strlen_start: ; this loop finds the length of the string as part of
   ↪ the putstring function
26
27 cmp [rbx],byte 0 ; compare byte at address rdx with 0
28 jz putstring_strlen_end ; if comparison was zero, jump to loop end because we
   ↪ have found the length
29 inc rbx
30 jmp putstring_strlen_start
31
32 putstring_strlen_end:
33 sub rbx,rax ;rbx will now have correct number of bytes
34
35 ;write string using Linux Write system call
36 ;https://www.chromium.org/chromium-os/developer-library/reference/linux-const
   ↪ ants/syscalls/#x86\_64-64-bit
37
38 mov rdx,rbx      ;number of bytes to write
39 mov rsi,rax      ;pointer/address of string to write
40 mov rdi,1        ;write to the STDOUT file
41 mov rax,1        ;invoke SYS_WRITE (kernel opcode 1 on 64 bit systems)
42 syscall          ;system call to write the message
43
44 pop rdx
45 pop rcx
46 pop rbx

```



```
47 pop rax
48
49 ret ; this is the end of the putstring function return to calling location
50
51
52 ; This Assembly source file has been formatted for the FASM assembler.
53 ; The following 3 commands assemble, give executable permissions, and run the
   ↪ program
54 ;
55 ; fasm main.asm
56 ; chmod +x main
57 ; ./main
```

You may notice that the 64-bit program also uses the `syscall` instruction rather than `interrupt 0x80`. On my machine both programs behave identically because both calling conventions are valid. There are executables that run in 32 bit mode and others that run in 64 bit mode. They are not usually compatible and the FASM assembler has to be told which format is being assembled.

FASM has been my preferred assembler for a long time because unlike NASM, it has everything it needs to create executables without depending on a linker.

“What is a linker?” You might be asking. You see, the developers of Linux never really expected for people to be writing applications entirely in assembly. Usually they are written in C and then GCC compiles it to assembly that only the Gnu assembler (informally called Gas) can assemble and then link with the standard library. There is a linker program called “ld” that GCC automatically uses.

However, through some research and experimentation, I have converted the previous 64 bit FASM program into the Gas syntax. As you read it, remember that the AT&T phone company made this weird alternative syntax. The source and destination have been flipped so you will see the register receiving data on the right side instead of the left.

main.s (GNU Assembler 64 bit)

```

1  # Using Linux System calls for 64-bit
2  # Tested with GNU Assembler on Debian 12 (bookworm)
3  # It uses Chastity's putstring function for output
4
5  .global _start
6
7  .text
8
9  _start:
10
11  mov $main_string,%rax # move address of string into rax register
12  call  putstring      # call the putstring function Chastity wrote
13  mov  $0x3c,%eax      # system call 60 is exit
14  mov  $0x0,%edi       # we want to return code 0
15  syscall              # end program with system call
16
17  main_string:
18  .string "This program runs in Linux!\n"
19
20  putstring:           # the start of the putstring function
21  push  %rax
22  push  %rbx
23  push  %rcx
24  push  %rdx
25  mov  %rax,%rbx
26
27  putstring_strlen_start:
28  cmpb  $0x0,(%rbx)
29  je    putstring_strlen_end
30  inc   %rbx
31  jmp   putstring_strlen_start
32
33  putstring_strlen_end:
34  sub   %rax,%rbx # subtract rax from rbx for number of bytes to write
35  mov   %rbx,%rdx # copy number of bytes from rbx to rdx
36  mov   %rax,%rsi # address of string to output
37  mov   $0x1,%edi # file handler 1 is stdout
38  mov   $0x1,%rax # system call 1 is write
39  syscall
40  pop   %rdx
41  pop   %rcx
42  pop   %rbx
43  pop   %rax
44  ret
45
46  # This Assembly source file has been formatted for the GNU assembler.
47  # The following makefile rule has commands to assemble, link, and run the
   ↪  program
48  #
49  #main-gas:

```

```
50 # gcc -nostdlib -nostartfiles -nodefaultlibs -static main.s -o main
51 # strip main
52 # ./main
```

Although I find the GNU Assembler syntax hard to read, the fact that this assembler exists as part of the GNU Compiler Collection means that it is usually available even on systems that don't have FASM or NASM available.

It is possible to use NASM also but it can't create executables and requires linking with "ld" anyway. It is better to just write directly for the GNU Assembler or stick with FASM if you prefer intel syntax.

However, the beauty is that the machine code bytes from both types of assembly are identical! In fact that is how I got the GAS version. I had to assemble the other version and then disassemble it with objdump to get the equivalent syntax.

The programs you saw in this chapter only work on Linux, but Linux is Free both in terms of Software Freedom and Free in price too because anyone with an internet connection can download the ISO of a new operating system and install it on their computer as long as they take the time to read directions from the makers of that distribution. In fact Debian, Arch, Gentoo, and FreeBSD (not Linux but very similar) all have great instruction manuals. If you have managed to read this book, then you will have no problem following their stuff.

Chapter 9: Bitwise Operations for Advanced Nerds

This chapter contains information which will assist you in understanding more about how computers work, but that in general is not required for MOST programming unless you are trying to operate on individual bits.

To start out, I will describe 5 essential bitwise operations independently of any specific programming language. This is because these operations exist in every programming language I know of, including Assembly and C.

After I have explained what the bitwise operations do, I will give examples of how this can be used in Assembly language to substitute for addition and subtraction! You might wonder why you would do this. The fact is that you don't need to but it is a fun trick that only advanced nerds like me do for a special challenge.

The Bitwise Operations

This chapter explains 5 bitwise operations which operate on the bits of data in a computer. For the purpose of demonstration, it doesn't matter which number the bits represent at the moment. This is because the bits don't have to represent numbers at all but can represent anything described in two states. Bits are commonly used to represent statements that are **true** or **false**. For the purposes of this section, the words AND, OR, XOR are in capital letters because their meaning is only loosely related to the English words they get their name from.

Bitwise AND Operation

```
1 0 AND 0 == 0
2 0 AND 1 == 0
3 1 AND 0 == 0
4 1 AND 1 == 1
```

Think of the bitwise AND operation as multiplication of single bits. 1 times 1 is always 1 but 0 times anything is always 0. That's how I personally think of it. I guess you could say that something is true only if two conditions are true. For example, if I go to Walmart AND do my job then it is true that I get paid.

I like to think of the AND operation as the “prefer 0” operation. It will always choose a 0 if either of the two bits is a 0, otherwise, if no 0 is available, it will choose 1.

Bitwise OR Operation

```
1 0 OR 0 == 0
2 0 OR 1 == 1
3 1 OR 0 == 1
4 1 OR 1 == 1
```

The bitwise OR operation can be thought of as something that is true if one or two conditions are true. For example, it is true that playing in the street will result in you dying because you got run over by a car. It is also true that if you live long enough, something else will kill you. Therefore, the bit of your impending death is always 1.

I like to think of the OR operation as the “prefer 1” operation. It will always choose a 1 if one of the two bits is a 1, otherwise, if no 1 is available, it will choose 0.

Bitwise XOR Operation

```
1 0 XOR 0 == 0
2 0 XOR 1 == 1
3 1 XOR 0 == 1
4 1 XOR 1 == 0
```

The bitwise XOR operation is different because it isn't really used much for evaluating true or false. Instead, this operation returns 1 if the bits compared

are different or 0 if they are the same. This means that any bit, or group of bits, XORed with itself, will always result in 0.

If you look at my XOR chart above, you will see that using XOR of any bit with a 1 causes the result to be the opposite of the original bit.

The XOR operation is the quickest way to achieve this bit inversion. If you have a setting that you want to switch on or off, you can toggle it by XORing that bit with 1.

While the AND, OR, XOR operations can work in the context of individual bits, or groups of them, the next operations, the bit shifts, only make sense in the context of a group of bits. At minimum, you will be operating on 8 bits at a time because a byte is the lowest addressable size of memory.

Bitwise Left and Right Shift Operations

Consider the case of the following 8 bit binary value:

00001000

This would of course represent the number 8 because a 1 is in the 8's place value. We can left shift or right shift.

```
1 00001000 == 8 : is the original byte
2
3 00010000 == 16 : original left shift 1
4 00000100 == 4 : original right shift 1
```

That is really all there is to shifts. They can be used to multiply or divide by a power of two. In some cases, this can be faster than using the mul and div instructions described in chapter 4.

Example 0: Fake Add

The following example shows how it is possible to write an addition routine using a combination of the AND,XOR,SHL operations. In this case, the numbers are shown in decimal to be easier for most people to see that the addition is correct.

```
1  org 100h
2
3  main:
4
5  mov word [radix],10 ; choose radix for integer input/output
6  mov word [int_width],1
7
8  mov di,1987
9  mov si,38
10
11 mov ax,di
12 call putint
13 mov ax,si
14 call putint
15 call putline
16
17 fake_add:
18 mov ax,di
19 xor di,si
20 and si,ax
21 shl si,1
22 jnz fake_add
23
24 mov ax,di
25 call putint
26 mov ax,si
27 call putint
28
29 mov ax,4C00h
30 int 21h
31
32 include 'chastelib16.asm'
```

If you run it, you will see that the correct result of 2025 which is 1987+38. These are the values we set the di and si registers to before simulating addition with these fancy bitwise operations that make even seasoned programmers run scared.

But how does this monstrosity of a program work? You see the AND operation keeps track of whether both bits in each place value are 1 or not. If they both are, this means that we have to “carry” those bits as we would do in an ordinary binary addition. We store the carry in the si register and then left shift it once each time in the loop. The loop continues until si equals zero and there are no more bits to invert with XOR.

The fact that it works is easy to work out in my head but I don’t blame you if you can’t visualize it. However, this shows the power of what bit operations

can do, even though you will probably never need to do this.

Example 1: Fake Sub

In case the fake addition example above wasn't enough for you, here is a slightly modified example that does a fake subtraction operation using the same operations. Try it out and you will see that it subtracts 38 from 2025 and gets the original 1987.

```
1  org 100h
2
3  main:
4
5  mov word [radix],10 ; choose radix for integer input/output
6  mov word [int_width],1
7
8  mov di,2025
9  mov si,38
10
11 mov ax,di
12 call putint
13 mov ax,si
14 call putint
15 call putline
16
17 fake_sub:
18 xor di,si
19 and si,di
20 shl si,1
21 jnz fake_sub
22
23 mov ax,di
24 call putint
25 mov ax,si
26 call putint
27
28 mov ax,4C00h
29 int 21h
30
31 include 'chastelib16.asm'
```

I will try to explain how this works. You see, we first XOR the di register with the si register. Then, we AND si with the new value of di. This means that the bits in the current place value will only both be 1 if those bits were 0 in di and then were inverted to 1 by the XOR with si. This means that at the start of

the loop, destination bit=0 and source bit=1. 0 minus 1 means that we need to “borrow” (I hate that term because it is really stealing because we never give it back). We left shift si as usual and then we keep XORing the new borrow in si until it is zero.

Also, you may have noticed that I never used the “cmp” instruction to compare si with zero in this examples. This is because the zero flag is automatically updated with most operations. In fact there are places in my standard library of functions (chastelib) where it wasn’t strictly required to compare with “cmp” but I added it for clarity so I could read my code and more easily remember what I was doing.

But let’s face it, the examples in this chapter are purely for showing off how advanced my knowledge of the binary numeral system and manipulating bits in ways no reasonable person should ever attempt. I must admit, it would be great for an obfuscated code contest to make a program with code that is unreadable to most humans.

Chapter 10: chastehex: Not just a program, but a philosophy

For the final program in this book, I have prepared something special. It is a command-line hex editor written entirely in assembly. It does not have a graphical user interface, but instead can be used to read or write bytes of a file at any location!

I will be showing you the full source code as one big file that you can copy or download directly from my GitHub repository. But first, I need to show you an example of how it works when it is assembled.

Starting from a DOSBox prompt, I create a small text file with this command:

```
echo brilliant > company.txt
```

That is literally the name of a company I used to work for. It is a good example to use here because the official spelling is wrong. Brilliant is a misspelling of brilliant, which means smart (unlike whoever chose the company name).

To use the chastehex (named chex.com in this example) program to view what is in the file, enter this command:

```
chex company.txt
```

You will see the following.

```
1 company.txt
2 00000000 62 72 69 6C 6C 69 65 6E 74 0D 0A          brilliant..
3 E0F
```

It is possible to change the 'e' into an 'a' and correct the spelling. We only need to change the hexcode for that byte.

The 'e' is the seventh letter in the word, which means it is address 6 because the first address of any file starts at 0. Knowing this, the following command does the trick:

```
chex company.txt 6 61
```

If you view the file again after this as explained earlier, it will show that it has been corrected.

```
1 company.txt
2 00000000 62 72 69 6C 6C 69 61 6E 74 0D 0A          brilliant..
3 EOF
```

Obviously this is a silly example because there are plenty of other ways to correct a typo in a word. However, the power of this program comes from the fact that it is dynamic enough to handle not just text files but binary files of any size less than 2 gigabytes. You can, in fact, edit executable files, game save files, image files, or anything where you know the precise location of bytes you need to change for some reason.

If you enter chex without any arguments, a short help message will display:

```
1 chastehex:
2 hexdump a file:
3     chex file
4 read a byte:
5     chex file address
6 write a byte:
7     chex file address value
8 The file must exist
```

The flexibility of this program comes from the fact that it changes behavior based on how many arguments you give it. It can be used to dump any entire file or to read and write individual bytes. If you add more than 3 arguments it will accept the numbers as values of more bytes to write at the location you selected.

I frequently use this program for messing around with files just for the fun of it. But besides being a fun toy for messing with binary files, it also serves as an example of how much can be accomplished with assembly language. All it does is process command line arguments, open, read/write, and close files while also displaying basic information to the screen on what it is doing.

The idea of how it behaves is easy to understand, but it also can be a bit complex to write such a program. The good news is you don't have to, because I wrote the entire program myself as an example of how much skill I have with Assembly language for DOS.

Below is the full source of chastehex for DOS. It will assemble with either the fasm or nasm assemblers.

chex.asm

```

1  org 100h      ;DOS programs start at this address
2
3  mov word [radix],16 ; can choose radix for integer output!
4
5  mov ch,0      ;zero ch (upper half of cx)
6  mov cl,[80h] ;load length in bytes of the command string
7  cmp cx,0
8  jnz args_exist
9
10 mov ax,help    ;if not arguments were given, show a help message
11 call putstring
12 jmp ending     ;and end the program because there is nothing to do
13
14 args_exist:
15
16 mov dx,81h     ;Point dx to the beginning of string
17 inc dx         ;go to next char
18 dec cx        ;but subtract 1 from count
19 mov [arg_index],dx ;save index to variable so dx is free to change as needed
20
21 ;find the end of the string based on length
22 mov ax,dx
23 add ax,cx
24 ;now we know where the string ends.
25 mov [arg_string_end],ax ;this is the end of the arg string. important for later
26 ;call putint ; print address where entire arg string ends
27
28 ;this routine replaces all non printable characters with zero in the arg string
29 mov bx,dx
30 filter:
31 cmp byte [bx], ' '
32 ja notspace ; if char is above space, leave it alone
33 mov byte [bx],0 ;otherwise it counts as a space, change it to a zero
34 notspace:
35 inc bx
36 cmp bx,[arg_string_end] ;are we at the end of the arg string?
37 jnz filter ;if not at end, continue the filter
38
39 filter_end:
40 mov byte [bx],0 ;terminate the ending with a zero for safety
41
42 ;now that the argument string is prepared, we will try to use the first
   ↪ argument as a filename to open

```

```
43
44 mov ah,3Dh          ;call number for DOS open existing file
45 mov al,2            ;file access: 0=read,1=write,2=read+write
46 mov dx,[arg_index] ;string address to interpret as filename
47 int 21h             ;DOS call to finalize open function
48
49 mov [file_handle],ax ;save the file handle
50
51 jc file_error ;if carry flag is set, we have an error, otherwise, file is open
52
53 file_opened:
54
55 mov ax,dx
56 call putstring
57 call putline
58 jmp use_file ;skip past error message and start using the file
59
60 ;this section prints error message and then ends the program if file error found
61
62 file_error: ;prints error code2=file not found
63 mov ax,dx
64 call putstring
65 call putline
66 mov ax,file_error_message
67 call putstring
68 mov ax,[file_handle]
69 call putint
70 jmp arg_loop_end
71
72 ;how we use the file depends on the number of arguments given
73 ;if no arguments other than the filename exist, we do a regular hex dump
74
75 use_file:
76
77 mov bp,0 ;set bp to zero because it represents upper 16 bits of file address
78 call get_next_arg ;get address of next arg and return into ax register
79 cmp ax,[arg_string_end] ;this time, if ax equals end of string, we hex dump and
80   ↳ then end the program later
81 jmp hexdump ;jump to hexdump section
82
83 ;otherwise, if there are more args, ax contains next argument
84 ;we will next extra the address from this argument for future operations
85
86 ;first call the strint_32 function to get 32 bit integer from a hex string
87 ;the lower 16 bits are stored in ax just like regular strint
88 ;upper 16 bits are stored in the "extra_word" memory location
89 ;but then I copy them to the bp register to use for the rest of the program
90 call strint_32
91 mov bp,[extra_word] ;store the upper 16 bits in the bp register
```

```

92 ;this number will be out new offset to seek to
93 mov [file_offset],ax
94
95 mov ah,42h          ;lseek call number
96 mov al,0            ;seek origin 00h start of file,01h current file
   ↪ position,02h end of file
97 mov bx,[file_handle]
98 mov cx,bp          ;upper word of offset
99 mov dx,[file_offset] ;lower word of offset
100 int 21h
101
102 jc arg_loop_end ;end program if seek error (though I can't imagine how it would
   ↪ fail)
103
104 ;check if there are any more args
105 call get_next_arg
106 cmp ax,[arg_string_end]
107 jz dump_byte ;jump to dump_byte section and continue with read mode
108
109 jmp arg_loop ;otherwise we jump to the arg loop and write the values as bytes
   ↪ starting at offset
110
111 ;this next section is the reading mode that reads one byte. It only executes if
   ↪ we have not provided bytes to write to the new address
112 ;because we have an argument for an address we will read only this byte and
   ↪ display it
113 dump_byte:
114
115 mov ah,3Fh          ;call number for read function
116 mov bx,[file_handle] ;store file handle to read from in bx
117 mov cx,1            ;we are reading only 1 byte
118 mov dx,byte_array   ;store the bytes here
119 int 21h
120
121 mov cx,ax ;number of bytes read
122
123 ;set width to 4 and display extra:offset
124 mov word[int_width],4
125 mov ax,bp
126 call putint
127 mov ax,[file_offset]
128 call putint_and_space
129
130 cmp cx,1
131 jz not_eof ;skip past here as long as one byte was read otherwise show EOF
132 mov ax,end_of_file
133 call putstring
134 jmp arg_loop_end
135 not_eof:
136

```

```
137 mov ah,0 ;zero upper half of ax
138 mov al,[byte_array]
139
140 mov word[int_width],2
141 call putint
142 call putline
143
144 jmp arg_loop_end ;we are done so we end the program
145
146 hexdump:
147
148 ;we start the loop with a call to read exactly 16 bytes
149
150 mov ah,3Fh ;call number for read function
151 mov bx,[file_handle] ;store file handle to read from in bx
152 mov cx,16 ;we are reading sixteen bytes
153 mov dx,byte_array ;store the bytes here
154 int 21h
155
156 ;call putint ;check the number of bytes read
157
158 ;important note: the number of bytes read should be 16 or less and this is not
    ↳ an error
159 ;zero is expected if we are at the end of the file.
160 ;however, if it is zero, we print an EOF message and exit
161
162 cmp ax,0
163 jnz hexdump_print_row
164 mov ax,end_of_file
165 call putstring
166 jmp arg_loop_end
167
168 hexdump_print_row:
169
170 mov [bytes_read],ax
171
172 call print_bytes_row
173
174 jmp hexdump ;jump back to hexdump and attempt another read of a row
175
176 ;this loop processes the rest of the arguments
177 ;it interprets each one as a byte to write to the current offset
178 ;this loop should only execute if a file name and address have already been
    ↳ given
179 arg_loop:
180 mov ax,[arg_index] ;get address of current arg
181 ;call putstring
182
183 call strint_32 ;turn string at address ax into a number returned in ax
184
```

```

185  mov [byte_array],al
186
187  mov ah,40h           ; select DOS function 40h write
188  mov bx,[file_handle] ;store file handle to write to in bx
189  mov cx,1             ;write 1 byte to this file
190  mov dx,byte_array    ;write from this address
191  int 21h
192
193  ;set width to 4 and display extra:offset
194  mov word[int_width],4
195  mov ax,bp
196  call putint
197  mov ax,[file_offset]
198  call putint_and_space
199
200  add word[file_offset],1
201  adc bp,0
202
203
204  mov word[int_width],2
205  mov ah,0
206  mov al,[byte_array]
207  call putint
208  call putline
209
210  call get_next_arg ;get address of next arg and return into ax register
211
212  cmp ax,[arg_string_end] ;if the ax register contains address of the end of args
   ↪ string, end program to avoid failure
213  jz arg_loop_end
214  jmp arg_loop
215
216  arg_loop_end: ;this is the correct end of the program
217
218  ;close the file if it is open
219  mov ah,3Eh
220  mov bx,[file_handle]
221  int 21h
222
223  ending:
224  mov ax,4C00h ; Exit program
225  int 21h
226
227  arg_string_end dw 0
228  arg_index dw 0
229  file_error_message db 'Could not open the file! Error number: ',0
230  file_handle dw 0
231  read_error_message db 'Failure during reading of file. Error number: ',0
232  end_of_file db 'EOF',0
233

```



```
234 ;where we will store data from the file
235 byte_array db 16 dup '?',0
236 file_offset dw 0,0
237 bytes_read dw 0
238
239 ;function to move ahead to the next arg
240 ;only works after the filter has been applied to turn all spaces into zeroes
241
242 get_next_arg:
243 mov bx,[arg_index] ;dx has address of current arg
244 find_zero:
245 cmp byte [bx],0
246 jz found_zero
247 inc bx
248 jmp find_zero ; this char is not zero, go to the next char
249 found_zero:
250
251 find_non_zero:
252 cmp bx,[arg_string_end]
253 jz arg_finish ;if bx is already at end, nothing left to find
254 cmp byte [bx],0
255 jnz arg_finish ;if this char is not zero we have found the next string!
256 inc bx
257 jmp find_non_zero ;otherwise, keep looking
258
259 arg_finish:
260 mov [arg_index],bx ; save this index to variable
261 mov ax,bx ;but also save it to ax register for use
262 ret
263
264 ;this function prints a row of hex bytes
265 ;each row is 16 bytes
266 print_bytes_row:
267 mov cx,[bytes_read] ;number of bytes read
268
269 ;set width to 4 and display extra:offset
270 mov word[int_width],4
271 mov ax,bp
272 call putint
273 mov ax,[file_offset]
274 call putint_and_space
275
276 add [file_offset],cx
277 adc bp,0
278
279 mov ah,0 ;zero upper half of ax
280 mov bx,byte_array
281
282 mov word[int_width],2
283
```

```

284 print_byte:
285 mov al,[bx]
286 call putint_and_space
287 inc bx
288 dec cx
289 cmp cx,0
290 jnz print_byte
291
292 ;optionally, print chars after hex bytes
293 call print_bytes_row_text
294 call putline
295
296 ret
297
298 ;I define a string of 3 spaces as filler when less than 16 bytes are read
299 ;This makes the text section on the right properly lined up.
300 space_three db '   ',0
301
302 ;This function prints the text equivalent of the bytes on the last row printed.
303 ;It reads how many bytes were read in the last read operation.
304 ;If less than 16 bytes were read, it prints spaces as filler so that
305 ;text can still be printed lined up with all the other rows
306 ;even if less than 16 bytes exist in the current row.
307 ;This situation sometimes happens when we get near the end of the file.
308 ;It also replaces characters that can't be printed with periods -> .
309
310 print_bytes_row_text:
311
312 mov cx,[bytes_read]
313 pad_spaces:
314 cmp cx,0x10
315 jz pad_spaces_end
316 mov ax,space_three
317 call putstring
318 inc cx
319 jmp pad_spaces
320 pad_spaces_end:
321
322 mov bx,byte_array
323 mov cx,[bytes_read]
324 next_char:
325 mov ax,0
326 mov al,[bx]
327
328 ;if char is below '0' or above '9', it is outside the range of these and is not
   ↪ a digit
329 cmp al,0x20
330 jb not_printable
331 cmp al,0x7E
332 ja not_printable

```

```

333
334 printable:
335 ;if char is in printable range,copy as is and proceed to next index
336 jmp next_index
337
338 not_printable:
339 mov al,'.' ;otherwise replace with placeholder value
340
341 next_index:
342 mov [bx],al
343 inc bx
344 dec cx
345 cmp cx,0
346 jnz next_char
347 mov [bx],byte 0 ;make sure string is zero terminated
348
349 mov ax,byte_array
350 call putstring
351
352 ret
353
354 help db 'chastehex:',0Dh,0Ah
355 db 'hexdump a file:',0Dh,0Ah,9,'chex file',0Dh,0Ah
356 db 'read a byte:',0Dh,0Ah,9,'chex file address',0Dh,0Ah
357 db 'write bytes:',0Dh,0Ah,9,'chex file address byte1 byte2 etc.',0Dh,0Ah
358 db 4 dup 0
359
360 ; About the chastelib variant
361
362 ;instead of including chastelib16.asm as a header file
363 ;I copy pasted it except that I excluded functions that were not used.
364 ;Notably, the strint function is excluded because strint_32 is used instead
365 ;This was to squeeze chastehex for DOS down to 1024 bytes.
366
367 ;start of chastelib
368
369 ; This file is where I keep my function definitions.
370 ; These are usually my string and integer output routines.
371
372 ;this is my best putstring function for DOS because it uses call 40h of
↪ interrupt 21h
373 ;this means that it works in a similar way to my Linux Assembly code
374 ;the plan is to make both my DOS and Linux functions identical except for the
↪ size of registers involved
375
376 putstring:
377
378 push ax
379 push bx
380 push cx

```

```

381 push dx
382
383 mov bx,ax                ;copy ax to bx for use as index register
384
385 putstring_strlen_start:  ;this loop finds the length of the string as part of
    ↪ the putstring function
386
387 cmp [bx], byte 0         ;compare this byte with 0
388 jz putstring_strlen_end  ;if comparison was zero, jump to loop end because we
    ↪ have found the length
389 inc bx                   ;increment bx (add 1)
390 jmp putstring_strlen_start ;jump to the start of the loop and keep trying until
    ↪ we find a zero
391
392 putstring_strlen_end:
393
394 sub bx,ax                ; sub ax from bx to get the difference for number of
    ↪ bytes
395 mov cx,bx                ; mov bx to cx
396 mov dx,ax                ; dx will have address of string to write
397
398 mov ah,40h               ; select DOS function 40h write
399 mov bx,1                 ; file handle 1=stdout
400 int 21h                  ; call the DOS kernel
401
402 pop dx
403 pop cx
404 pop bx
405 pop ax
406
407 ret
408
409 ;this is the location in memory where digits are written to by the intstr
    ↪ function
410 int_string db 16 dup '?' ;enough bytes to hold maximum size 16-bit binary
    ↪ integer
411 int_string_end db 0 ;zero byte terminator for the integer string
412
413 radix dw 2 ;radix or base for integer output. 2=binary, 8=octal, 10=decimal,
    ↪ 16=hexadecimal
414 int_width dw 8
415
416 intstr:
417
418 mov bx,int_string_end-1 ;find address of lowest digit(just before the newline
    ↪ 0Ah)
419 mov cx,1
420
421 digits_start:
422

```

```

423  mov dx,0;
424  div word [radix]
425  cmp dx,10
426  jb decimal_digit
427  jnb hexadecimal_digit
428
429  decimal_digit: ;we go here if it is only a digit 0 to 9
430  add dx,'0'
431  jmp save_digit
432
433  hexadecimal_digit:
434  sub dx,10
435  add dx,'A'
436
437  save_digit:
438
439  mov [bx],dl
440  cmp ax,0
441  jz intstr_end
442  dec bx
443  inc cx
444  jmp digits_start
445
446  intstr_end:
447
448  prefix_zeros:
449  cmp cx,[int_width]
450  jnb end_zeros
451  dec bx
452  mov [bx],byte '0'
453  inc cx
454  jmp prefix_zeros
455  end_zeros:
456
457  mov ax,bx ; store string in ax for display later
458
459  ret
460
461  ;function to print string form of whatever integer is in ax
462  ;The radix determines which number base the string form takes.
463  ;Anything from 2 to 36 is a valid radix
464  ;in practice though, only bases 2,8,10,and 16 will make sense to other
    ↪ programmers
465  ;this function does not process anything by itself but calls the combination of
    ↪ my other
466  ;functions in the order I intended them to be used.
467
468  putint:
469
470  push ax

```

```
471 push bx
472 push cx
473 push dx
474
475 call intstr
476 call putstring
477
478 pop dx
479 pop cx
480 pop bx
481 pop ax
482
483 ret
484
485 ;the next utility functions simply print a space or a newline
486 ;these help me save code when printing lots of things for debugging
487
488 space db ' ',0
489 line db 0Dh,0Ah,0
490
491 putspace:
492 push ax
493 mov ax,space
494 call putstring
495 pop ax
496 ret
497
498 putline:
499 push ax
500 mov ax,line
501 call putstring
502 pop ax
503 ret
504
505 ;a small function just for the common operation
506 ;printing an integer followed by a space
507 ;this saves a few bytes in the assembled code
508
509 putint_and_space:
510 call putint
511 call putspace
512 ret
513
514 ;end of chastelib
515
516 ; About the strint_32 function
517
518 ;this function converts a string pointed to by ax into an integer returned in
    ↪ ax instead
```

```

519 ;it is a little complicated because it has to account for whether the character
    ↪ in
520 ;a string is a decimal digit 0 to 9, or an alphabet character for bases higher
    ↪ than ten
521 ;it also checks for both uppercase and lowercase letters for bases 11 to 36
522 ;finally, it checks if that letter makes sense for the base.
523 ;For example, G to Z cannot be used in hexadecimal, only A to F can
524 ;The purpose of writing this function was to be able to accept user input as
    ↪ integers
525
526 ;this version of the strint function has been modified from its original
    ↪ version.
527 ;it has been formatted to extract up to 32 bits of data using memory despite
    ↪ using
528 ;only 16 bit registers.
529 ;It uses the same [radix] and [int_width] variables as the regular 16 bit strint
530
531 ;However, it uses an extra word variable in memory which is designed to store
    ↪ the upper 16 bits of
532 ;a 32 bit offset for file seeking. Apparently the DOS system calls support this
    ↪ based on Ralf Browns interrupt list.
533 ;I confirmed that it works by writing to higher addresses and reading them
534
535 ;You might wonder why I made a 32 bit variant of this function rather than
    ↪ replacing the original. These are my reasons
536
537 ;1. It only works with hexadecimal in the context of the chastehex program.
538 ;2. This function stores extra data every loop and is therefore slower.
539 ;3. Most of the time 32 bit data isn't needed as 16 bit DOS can't use 32 bit
    ↪ memory.
540
541 ;This function was a specific case only meant for adding 32 bit support for the
    ↪ DOS version of chastehex.
542 ;I also kept the original which only contains support for files less than 64
    ↪ kilobytes.
543
544 extra_word dw 0 ;define an extra word(16 bits). The initial value doesn't
    ↪ matter.
545
546 strint_32:
547
548 ;initialize new variables added to this function
549 mov word[extra_word],0
550
551 mov bx,ax ;copy string address from ax to bx because ax will be replaced soon!
552 mov ax,0
553
554 read_strint_32:
555 mov cx,0 ; zero ecx so only lower 8 bits are used
556 mov cl,[bx]

```

```
557 inc bx
558 cmp cl,0 ; compare byte at address edx with 0
559 jz strint_end_32 ; if comparison was zero, this is the end of string
560
561 ;if char is below '0' or above '9', it is outside the range of these and is not
    ↳ a digit
562 cmp cl,'0'
563 jb not_digit_32
564 cmp cl,'9'
565 ja not_digit_32
566
567 ;but if it is a digit, then correct and process the character
568 is_digit_32:
569 sub cl,'0'
570 jmp process_char_32
571
572 not_digit_32:
573 ;it isn't a digit, but it could be perhaps an alphabet character
574 ;which is a digit in a higher base
575
576 ;if char is below 'A' or above 'Z', it is outside the range of these and is not
    ↳ capital letter
577 cmp cl,'A'
578 jb not_upper_32
579 cmp cl,'Z'
580 ja not_upper_32
581
582 is_upper_32:
583 sub cl,'A'
584 add cl,10
585 jmp process_char_32
586
587 not_upper_32:
588
589 ;if char is below 'a' or above 'z', it is outside the range of these and is not
    ↳ lowercase letter
590 cmp cl,'a'
591 jb not_lower_32
592 cmp cl,'z'
593 ja not_lower_32
594
595 is_lower_32:
596 sub cl,'a'
597 add cl,10
598 jmp process_char_32
599
600 not_lower_32:
601
602 ;if we have reached this point, result invalid and end function
603 jmp strint_end_32
```



```
604
605 process_char_32:
606
607 cmp cx,[radix] ;compare char with radix
608 jnb strint_end_32 ;if this value is above or equal to radix, it is too high
   ↳ despite being a valid digit/alpha
609
610 ;before we process the character, to avoid data loss, we shift bits into the
   ↳ [extra_word]
611 push ax
612 shr ax,12 ;shift exactly 12 bits to keep the lowest hex digit of ax
613 shl word[extra_word],4 ;shift the [extra_word] 4 bits to make room for the hex
   ↳ digit
614 add [extra_word],ax
615 pop ax
616
617 mov dx,0 ;zero edx because it is used in mul sometimes
618 mul word [radix] ;mul ax with radix
619 add ax,cx
620
621 jmp read_strint_32 ;jump back and continue the loop if nothing has exited it
622
623 strint_end_32:
624
625 ret
```

The chastehex program is a massive beast of Assembly code. It is a 625 line masterpiece that has been tested under the traditional DOSBox emulator and the newer DOSBox-X.

I admit that it is a little hard to understand because Assembly source takes more space and it can be easier to get lost in it.

But the key reason I bothered is efficiency and frugality of the final machine code. Despite the large source code size of chastehex, the assembled binary is only 1024 bytes. Here it is:

chex.com

```

1  chex.com
2  00000000 C7 06 28 04 10 00 B5 00 8A 0E 80 00 83 F9 00 75 ..(.....u
3  00000010 09 B8 75 03 E8 E0 02 E9 42 01 BA 81 00 42 49 89 ..u....B....BI.
4  00000020 16 63 02 89 D0 01 C8 A3 61 02 89 D3 80 3F 20 77 .c.....a....? w
5  00000030 03 C6 07 00 43 3B 1E 61 02 75 F1 C6 07 00 B4 3D ....C;.a.u.....=
6  00000040 B0 02 8B 16 63 02 CD 21 A3 8D 02 72 0A 89 D0 E8 ....c...!...r....
7  00000050 A5 02 E8 2E 03 EB 17 89 D0 E8 9B 02 E8 24 03 B8 .....$..
8  00000060 65 02 E8 92 02 A1 8D 02 E8 FB 02 E9 E6 00 BD 00 e.....
9  00000070 00 E8 65 01 3B 06 61 02 74 6B E8 18 03 8B 2E 93 ..e.;.a.tk.....
10 00000080 04 A3 D3 02 B4 42 B0 00 8B 1E 8D 02 89 E9 8B 16 .....B.....
11 00000090 D3 02 CD 21 0F 82 BC 00 E8 3E 01 3B 06 61 02 74 ...!.....>;.a.t
12 000000A0 02 EB 65 B4 3F 8B 1E 8D 02 B9 01 00 BA C2 02 CD ..e.?.....
13 000000B0 21 89 C1 C7 06 2A 04 04 00 89 E8 E8 A8 02 A1 D3 !....*.....
14 000000C0 02 E8 C8 02 83 F9 01 74 09 B8 BE 02 E8 28 02 E9 .....t.....(..
15 000000D0 82 00 B4 00 A0 C2 02 C7 06 2A 04 02 00 E8 86 02 .....*.....
16 000000E0 E8 A0 02 EB 6F B4 3F 8B 1E 8D 02 B9 10 00 BA C2 ....o.?.....
17 000000F0 02 CD 21 83 F8 00 75 08 B8 BE 02 E8 F9 01 EB 54 ..!...u.....T
18 00000100 A3 D7 02 E8 F4 00 EB DD A1 63 02 E8 87 02 A2 C2 .....c.....
19 00000110 02 B4 40 8B 1E 8D 02 B9 01 00 BA C2 02 CD 21 C7 ..@.....!..
20 00000120 06 2A 04 04 00 89 E8 E8 3C 02 A1 D3 02 E8 5C 02 .*.....<.....\..
21 00000130 83 06 D3 02 01 83 D5 00 C7 06 2A 04 02 00 B4 00 .....*.....
22 00000140 A0 C2 02 E8 20 02 E8 3A 02 E8 8D 00 3B 06 61 02 .... ..;.....a.
23 00000150 74 02 EB B4 B4 3E 8B 1E 8D 02 CD 21 B8 00 4C CD t....>.....!..L.
24 00000160 21 00 00 00 00 43 6F 75 6C 64 20 6E 6F 74 20 6F !....Could not o
25 00000170 70 65 6E 20 74 68 65 20 66 69 6C 65 21 20 45 72 pen the file! Er
26 00000180 72 6F 72 20 6E 75 6D 62 65 72 3A 20 00 00 00 46 ror number: ...F
27 00000190 61 69 6C 75 72 65 20 64 75 72 69 6E 67 20 72 65 ailure during re
28 000001A0 61 64 69 6E 67 20 6F 66 20 66 69 6C 65 2E 20 45 ading of file. E
29 000001B0 72 72 6F 72 20 6E 75 6D 62 65 72 3A 20 00 45 4F rror number: .E0
30 000001C0 46 00 3F 3F 3F 3F 3F 3F 3F 3F 3F 3F 3F 3F 3F F.?????????????
31 000001D0 3F 3F 00 00 00 00 00 00 00 8B 1E 63 02 80 3F 00 ??.....c.?.
32 000001E0 74 03 43 EB F8 3B 1E 61 02 74 08 80 3F 00 75 03 t.C.;.a.t..?..u.
33 000001F0 43 EB F2 89 1E 63 02 89 D8 C3 8B 0E D7 02 C7 06 C....c.....
34 00000200 2A 04 04 00 89 E8 E8 5D 01 A1 D3 02 E8 7D 01 01 *......}. ....}..
35 00000210 0E D3 02 83 D5 00 B4 00 BB C2 02 C7 06 2A 04 02 .....*..
36 00000220 00 8A 07 E8 66 01 43 49 83 F9 00 75 F4 E8 08 00 ....f.CI...u....
37 00000230 E8 50 01 C3 20 20 20 00 8B 0E D7 02 83 F9 10 74 .P.. .....t
38 00000240 09 B8 34 03 E8 B0 00 41 EB F2 BB C2 02 8B 0E D7 ..4....A.....
39 00000250 02 B8 00 00 8A 07 3C 20 72 06 3C 7E 77 02 EB 02 .....< r.<~w...
40 00000260 B0 2E 88 07 43 49 83 F9 00 75 E6 C6 07 00 B8 C2 ....CI...u.....
41 00000270 02 E8 83 00 C3 63 68 61 73 74 65 68 65 78 3A 0D ....chastehex:.
42 00000280 0A 68 65 78 64 75 6D 70 20 61 20 66 69 6C 65 3A .hexdump a file:
43 00000290 0D 0A 09 63 68 65 78 20 66 69 6C 65 0D 0A 72 65 ...chex file..re
44 000002A0 61 64 20 61 20 62 79 74 65 3A 0D 0A 09 63 68 65 ad a byte:...che
45 000002B0 78 20 66 69 6C 65 20 61 64 64 72 65 73 73 0D 0A x file address..
46 000002C0 77 72 69 74 65 20 62 79 74 65 73 3A 0D 0A 09 63 write bytes:...c
47 000002D0 68 65 78 20 66 69 6C 65 20 61 64 64 72 65 73 73 hex file address
48 000002E0 20 62 79 74 65 31 20 62 79 74 65 32 20 65 74 63 byte1 byte2 etc
49 000002F0 2E 0D 0A 00 00 00 00 50 53 51 52 89 C3 80 3F 00 .....PSQR...?.
50 00000300 74 03 43 EB F8 29 C3 89 D9 89 C2 B4 40 BB 01 00 t.C..).....@...

```

51	00000310	CD 21 5A 59 5B 58 C3 3F 3F 3F 3F 3F 3F 3F 3F 3F 3F	!.ZY[X.?????????
52	00000320	3F 3F 3F 3F 3F 3F 3F 00 02 00 08 00 BB 26 04 B9	???????,....&..
53	00000330	01 00 BA 00 00 F7 36 28 04 83 FA 0A 72 02 73 05	6(....r.s
54	00000340	83 C2 30 EB 06 83 EA 0A 83 C2 41 88 17 83 F8 00	..0.....A.....
55	00000350	74 04 4B 41 EB DC 3B 0E 2A 04 73 07 4B C6 07 30	t.KA.;.*.s.K..0
56	00000360	41 EB F3 89 D8 C3 50 53 51 52 E8 BF FF E8 87 FF	A....PSQR.....
57	00000370	5A 59 5B 58 C3 20 00 0D 0A 00 50 B8 75 04 E8 76	ZY[X.P.u.v
58	00000380	FF 58 C3 50 B8 77 04 E8 6D FF 58 C3 E8 D7 FF E8	.X.P.w..m.X.....
59	00000390	E8 FF C3 00 00 C7 06 93 04 00 00 89 C3 B8 00 00	
60	000003A0	B9 00 00 8A 0F 43 80 F9 00 74 54 80 F9 30 72 0A	C...tT..0r.
61	000003B0	80 F9 39 77 05 80 E9 30 EB 26 80 F9 41 72 0D 80	..9w...0.&..Ar.
62	000003C0	F9 5A 77 08 80 E9 41 80 C1 0A EB 14 80 F9 61 72	.Zw...A.....ar
63	000003D0	0D 80 F9 7A 77 08 80 E9 61 80 C1 0A EB 02 EB 1F	...zw...a.....
64	000003E0	3B 0E 28 04 73 19 50 C1 E8 0C C1 26 93 04 04 01	;(.s.P....&....
65	000003F0	06 93 04 58 BA 00 00 F7 26 28 04 01 C8 EB A1 C3	...X....&(.....
66	EOF		

In case you were wondering, that hex dump was produced by chastehex itself! This program has become one of my primary debugging tools when I am writing assembly language because it helps me see the exact bytes in the executable files or in the text files my programs write.

As it turns out, the core chastelib series of functions (putstring, intstr, putint, and strint) were written so that I could port the original C version I wrote on Linux. The program is unique in that it uses the closest DOS equivalent of the 6 POSIX functions read, write, open, close, lseek, and exit.

This meant that I first had to write the Linux Assembly version and construct my 4 functions. When the 32-bit Linux version was complete, the next step was to gradually port all the functions to DOS. I had to learn the system call numbers and translate the Linux calls into DOS calls.

I ran into trouble because DOS handles command line arguments differently than Linux does. However, I finally got the same behavior from the DOS version as the Linux version had.

It is not an exaggeration to say that I spend hundreds of hours on this program. In fact it took longer to write the comments for explaining it than it did to write the program and test it.

The reason I say chastehex is more than a program is because it follows my philosophy of how code should be written. It is smaller and faster than any assembly code that a C compiler can produce. It is also original enough that it could not be written by AI and still be this dense and efficient. Although I have written this same program for Linux in both Assembly and C forms, the

DOS version remains the one that I am most proud of because it is my highest achievement on the first operating system I ever used.

However, a program only good when people can understand what it does, and how to use it. A full understanding of a program comes from its source code. That's why this entire book was written to help people learn Assembly language and appreciate programs like chastehex. With the skills you learned, you may even write more impressive tools for DOS and other operating systems. If you become better than me, I have succeeded as a teacher!

I do hope that you have enjoyed this book as I attempted to teach some of the secrets of how DOS programs work at the assembly language level. I truly love and understand math at a different level than most people but I do hope to receive feedback for future editions of this book, including the Linux edition that I want to write in the future.

If you understood this book, congratulations, you are brilliant! If not, perhaps a more general introduction to programming in C is more at your skill level. See my free website version of my other programming book: Chastity's Code Cookbook.

<https://chastitywhiterose.github.io/Chastity-Code-Cookbook/>

Chapter 11: Bonus Content

When I wrote Assembly Arithmetic Algorithms for DOS, I had planned for chapter 10 to be the final chapter. However, it seemed like a good idea to include the original source code for the C version of chastehex. Although this book is not about the C programming language, C programmers can gain insight from seeing the original 150 lines of code and comments of the main source file compared to the Assembly version.

I also want my readers to have a useful program that they can compile and run on any operating system or architecture. It is with great pleasure that I present the portable ANSI C version of chastehex!

chastehex main.c

```
1  /*
2  This is the original C version of chastehex upon which the Assembly versions
   ↳ were based.
3  */
4
5  #include <stdio.h>
6  #include <stdlib.h>
7  #include "chastelib.h"
8
9  FILE* fp; /*file pointer*/
10 char bytes[17]; /*the byte buffer for hex and text dumping*/
11 int count=1; /*keeps track of how many bytes were read during each row*/
12
13 /*outputs the ASCII text to the right of the hex field*/
14 void textdump()
15 {
16     int a,x=0;
17
18     x=count;
19     while(x<0x10)
20     {
21        _putstr("    ");
22         x++;
23     }
24
```

```
25  x=0;
26  while(x<count)
27  {
28      a=bytes[x];
29      if( a < 0x20 || a > 0x7E ){a='.';bytes[x]=a;}
30      x++;
31  }
32  bytes[x]=0;
33
34 _putstr(bytes);
35  }
36
37  /*outputs up to 16 bytes on each row in hexadecimal*/
38
39  void hexdump()
40  {
41      int x,address=0;
42      x=0;
43      while((count=fread(bytes,1,16,fp)))
44      {
45          int_width=8;
46          putint(address);
47         _putstr(" ");
48
49          int_width=2;
50          x=0;
51          while(x<count)
52          {
53              putint(bytes[x]&0xFF);
54             _putstr(" ");
55              x++;
56          }
57          textdump();
58         _putstr("\n");
59
60          address+=count;
61      }
62     _putstr("EOF\n");
63  }
64
65  int main(int argc, char *argv[])
66  {
67      int argx,x,c;
68
69      radix=0x10; /*set radix for integer output*/
70      int_width=1; /*set default integer width*/
71
72      if(argc==1)
73      {
74         _putstr
```

```

75     (
76         "Welcome to chastehex! The tool for reading and writing bytes of a file!\n\n"
77         "To hexdump an entire file:\n\n\tchastehex file\n\n"
78         "To read a single byte at an address:\n\n\tchastehex file address\n\n"
79         "To write a single byte at an address:\n\n\tchastehex file address value\n\n"
80     );
81     return 0;
82 }
83
84 if(argc>1)
85 {
86     fp=fopen(argv[1],"rb+");
87     if(fp==NULL)
88     {
89         putstr(argv[1]);
90         putstr("\nFailed to open file\n");
91         return 1;
92     }
93     else
94     {
95         putstr(argv[1]);
96         putstr("\n");
97     }
98 }
99
100 if(argc==2)
101 {
102     hexdump(); /*hex dump only if filename given*/
103 }
104
105 if(argc>2)
106 {
107     x=strint(argv[2]); /*extract a number from the argument string after the
108     ↪ filename*/
109     fseek(fp,x,SEEK_SET); /*seek to the address given in argument*/
110 }
111 /*read a byte at address of second arg*/
112 if(argc==3)
113 {
114     c=fgetc(fp);
115     int_width=8;
116     putint(x);
117     putstr(" ");
118     if(c==EOF){putstr("EOF");}
119     else
120     {
121         int_width=2;
122         putint(c);
123     }

```

```
124    _putstr("\n");
125 }
126
127 /*any arguments past the address are hex bytes to be written*/
128 if(argc>3)
129 {
130     argx=3;
131     while(argx<argc)
132     {
133         c=strint(argv[argx]);
134         int_width=8;
135         putint(x);
136        _putstr(" ");
137         int_width=2;
138         putint(c);
139        _putstr("\n");
140         fputc(c,fp);
141         x++;
142         argx++;
143     }
144 }
145
146 fclose(fp);
147 return 0;
148 }
149
150 /* gcc -Wall -ansi -pedantic main.c -o chastehex */
```

Of course, to compile the main.c file, you will need the C version of my standard library. For the most part, these functions are the same as the Assembly versions of the functions presented in this book. However, since C code can't refer to register names, variable names were chosen based on how they were used. For example, "s" is usually a string, "c" is a character, and "count" is used for the number of bytes used in the fwrite function of the C standard library as part of the putstring function.

chastelib.h


```

1  /*
2  This file is a C library of functions written by Chastity White Rose. The
   ↳ functions are for converting strings into integers and integers into
   ↳ strings.
3  I did it partly for future programming plans and also because it helped me
   ↳ learn a lot in the process about how pointers work
4  as well as which features the standard library provides, and which things I
   ↳ need to write my own functions for.
5
6  As it turns out, the integer output routines for C are too limited for my
   ↳ tastes. This library corrects this problem.
7  Using the global variables and functions in this file, integers can be output
   ↳ in bases/radixes 2 to 36.
8
9  Although this code is commented, I have also written a readme.md file designed
   ↳ to explain the usage of these functions and the philosophy behind them.
10 */
11
12 /*
13 These following lines define a static array with a size big enough to store
   ↳ the digits of an integer, including padding it with extra zeroes.
14 The integer conversion function (intstr) always references a pointer to this
   ↳ global string, and this allows other C standard library functions
15 such as printf to display the integers to standard output or even possibly to
   ↳ files.
16 This string can be repurposed for absolutely anything I desire.
17 */
18
19
20 #define usl 0x100 /*usl stands for Unsigned or Universal String Length.*/
21 char int_string[usl+1]; /*global string which will be used to store string of
   ↳ integers. Size is usl+1 for terminating zero*/
22
23 /*radix or base for integer output. 2=binary, 8=octal, 10=decimal,
   ↳ 16=hexadecimal*/
24 int radix=2;
25 /*default minimum digits for printing integers*/
26 int int_width=1;
27
28 /*
29 The intstr function is one that I wrote because the standard library can
   ↳ display integers as decimal, octal, or hexadecimal, but not any other
   ↳ bases(including binary, which is my favorite).
30
31 My function corrects this, and in my opinion, such a function should have been
   ↳ part of the standard library, but I'm not complaining because now I have my
   ↳ own, which I can use forever!
32 More importantly, it can be adapted for any programming language in the world
   ↳ if I learn the basics of that language. That being said, C is the best
   ↳ language and I will use it forever.

```

```

33  */
34
35  char *intstr(unsigned int i)    /*Chastity's supreme integer to string
    ↪ conversion function*/
36  {
37      int width=0;                /*the width or how many digits including
    ↪ prefixed zeros are printed*/
38      char *s=int_string+usl;    /*a pointer starting to the place where we will
    ↪ end the string with zero*/
39      *s=0;                      /*set the zero that terminates the string in the
    ↪ C language*/
40      while(i!=0 || width<int_width) /*loop to fill the string with every required
    ↪ digit plus prefixed zeros*/
41      {
42          s--;                    /*decrement the pointer to go left for correct
    ↪ digit placing*/
43          *s=i%radix;             /*get the remainder of division by the radix or
    ↪ base*/
44          i/=radix;              /*divide the input by radix*/
45          if(*s<10){*s+='0';}    /*convert digits 0 to 9 to the ASCII character
    ↪ for that digit*/
46          else{*s=*s+'A'-10;}    /*for digits higher than 9, convert to letters
    ↪ starting at A*/
47          width++;              /*increment the width so we know when enough
    ↪ digits are saved*/
48      }
49      return s;                  /*return this string to be used by
    ↪ putstr,printf,std::cout or whatever*/
50  }
51
52  /*
53  This function prints a string using fwrite.
54  This algorithm is the best C representation of how my Assembly programs also
    ↪ work.
55  Its true purpose is to be used in the putint function for conveniently
    ↪ printing integers,
56  but it can print any valid string.
57  */
58
59  int putstring(const char *s)
60  {
61      int count=0;                /*used to count how many bytes will be written*/
62      const char *p=s;            /*pointer used to find terminating zero of string*/
63      while(*p){p++;}            /*loop until zero found and immediately exit*/
64      count=p-s;                 /*count is the difference of pointers p and s*/
65      fwrite(s,1,count,stdout);  /*https://cppreference.com/w/c/io/fwrite.html*/
66      return count;              /*return how many bytes were written*/
67  }
68
69  /*

```

```

70  A function pointer named putstr which is a shorter name for calling putstring
71  But this doesn't exist just to save bytes of source files. Otherwise I
    ↳ wouldn't have these huge comments!
72  This exists so that all strings can be redirected to another function for
    ↳ output.
73  For example, if the strings were written to a log file during a game which
    ↳ didn't use a terminal.
74
75  But the most common use case is "putstr=addstr" when using the ncurses library
    ↳ to manage
76  terminal control functions for a text based game. Having the putstr pointer
    ↳ allows me to
77  include this same source file and use it for ncurses based projects.
78  */
79  int (*putstr)(const char *)=putstring;
80
81  /*
82  This function uses both intstr and putstring to print an integer in the
    ↳ currently selected radix and width.
83  */
84
85  void putint(unsigned int i)
86  {
87      putstr(intstr(i));
88  }
89
90  /*
91  The strint_errors variable is used to keep track of how many errors happened in
    ↳ the strint function.
92  The following errors can occur:
93
94  Radix is not in range 2 to 36
95  Character is not a number 0 to 9 or alphabet A to Z (in either case)
96  Character is alphanumeric but is not valid for current radix
97
98  If any of these errors happen, error messages are printed to let the programmer
    ↳ or user know what went wrong in the string that was passed to the function.
99  If getting input from the keyboard, the strint_errors variable can be used in a
    ↳ conditional statement to tell them to try again and recall the code that
    ↳ grabs user input.
100 */
101
102 int strint_errors = 0;
103
104 /*
105 The strint function is my own replacement for the strtol function from the C
    ↳ standard library.
106 I didn't technically need to make this function because the functions from
    ↳ stdlib.h can already convert strings from bases 2 to 36 into integers.

```

```

107  However, my function is simpler because it only requires 2 arguments instead
    ↪ of three, and it also does not handle negative numbers.
108  I have never needed negative integers, but if I ever do, I can use the standard
    ↪ functions or write my own in the future.
109  */
110
111  int strint(const char *s)
112  {
113      int i=0;
114      char c;
115      strint_errors = 0; /*set zero errors before we parse the string*/
116      if( radix<2 || radix>36 ){ strint_errors++; printf("Error: radix %i is out of
    ↪ range!\n",radix);}
117      while( *s == ' ' || *s == '\n' || *s == '\t' ){s++;} /*skip whitespace at
    ↪ beginning*/
118      while(*s!=0)
119      {
120          c=*s;
121          if( c >= '0' && c <= '9' ){c-='0';}
122          else if( c >= 'A' && c <= 'Z' ){c-='A';c+=10;}
123          else if( c >= 'a' && c <= 'z' ){c-='a';c+=10;}
124          else if( c == ' ' || c == '\n' || c == '\t' ){break;}
125          else{ strint_errors++; printf("Error: %c is not an alphanumeric
    ↪ character!\n",*s);break;}
126          if(c>=radix){ strint_errors++; printf("Error: %c is not a valid character for
    ↪ radix %i\n",*s,radix);break;}
127          i*=radix;
128          i+=c;
129          s++;
130      }
131      return i;
132  }
133
134
135  /*
136  Those four functions above are the core of chastelib.
137  While there may be extensions written for specific programs, these functions
    ↪ are essential for absolutely every program I write.
138
139  The only reason you would not need them is if you only output numbers in
    ↪ decimal or hexadecimal, because printf in C can do all that just fine.
140  However, the reason my core functions are superior to printf is that printf
    ↪ and its family of functions require the user to memorize all the arcane
    ↪ symbols for format specifiers.
141
142  The core functions are primarily concerned with standard output and the
    ↪ conversion of strings and integers. They do not deal with input from the
    ↪ keyboard or files. A separate extension will be written for my programs
    ↪ that need these features.
143  */

```

If you come from a background of C programming, you may appreciate how much more readable the C version of the program is compared to the Assembly code. I will not deny that writing Assembly code takes longer and is harder for most people to understand.

However, most of my C programs were the initial prototypes before I wrote the Assembly versions of the same thing. I do this because the C Programming Language provides maximum portability and readability.

But, Assembly language provides the maximum efficiency and control of hardware that I require for my own satisfaction. By using the right combination of C and Assembly, I have the best of both worlds:

1. C programs available for all devices and operating systems.
2. Assembly programs optimized for the specific device or operating system that I am using.

There are other tools I have written besides `chastehex`. However, at this time, `chastehex` is the only one I have tested and improved enough to be confident that I am providing quality code that can be used, studied, shared, and modified.

When I said that `chastehex` is not just a program, but a philosophy, I mean that it embodies the four freedoms outlined in the GNU General Public License and also my philosophy of well-written code.

I believe that computer programming is a task for humans to tell computers what we want done. I teach people so that, like me, they can write better software, in any language, and not depend on software vendors who may decide to fix bugs in the software you purchased from them if you bribe them with enough money.

In the modern world, I view code as a fundamental human right because our lives depend on software that works with our hardware and does not work against us and our right to privacy and freedom of speech.

Chapter Z: More Documentation

Below is a list of the sources I referenced the most while writing this book. I respect the work of Ralf Brown and any other people involved in keeping DOS programming information available.

<https://www.cs.cmu.edu/~ralf/files.html>
<https://www.delorie.com/djgpp/doc/rbinter/ix/>
https://stanislavs.org/helppc/int_21.html
<https://www.ctyme.com/intr/int-21.htm>

However, as time goes on, DOS information will become harder to find because old people die and can no longer pay to keep their websites online. This book was my attempt at keeping the information alive as long as I live. I have downloaded as much information onto my computer and have old books that are out of print. The time may come when I am the last person on earth who even knows or cares about the old way of programming in DOS.

And when I die, my only hope is that there is another young autistic programmer who will read my books about computer programming and Chess. May they be inspired to carry on the work of nerdy activities that most will never understand and criticize them for.

If at any time, something I wrote in this book is unclear to you, please email me to help me explain it better for you in future updates to this and other books.

chastitywhiterose@gmail.com

Appendix of System Calls for DOS

The following Interrupts are hand picked by Chastity for their usefulness in reading and writing characters in text based DOS programs. Most, but not all of these have already been used in this book. This does not cover BIOS calls for moving the console cursor, changing color of text, or changing video modes.

These were originally copied from the files "INTERRUP.F" in Ralf Brown's Interrupt List. However, the formatting was not compatible with Markdown and so I have made some effort to make it readable on modern devices that certainly didn't exist when Ralf Brown was alive and DOS was in common usage. This information is essential for knowing which numbers to put in which registers.

D-2100-TERMINATE PROGRAM

INT 21 - DOS 1+ - TERMINATE PROGRAM AH = 00h CS = PSP segment

Although this call will often end the program, it does not return a value back to the operating system like INT 21/AH=4Ch does. However, it can save a few bytes when trying to make the smallest .com files, so it is worth mentioning.

D-2101-READ CHARACTER

INT 21 - DOS 1+ - READ CHARACTER FROM STANDARD INPUT, WITH ECHO

1 AH = 01h

Return: AL = character read

This could be used to read characters one at a time for reading a string.

D-2102-WRITE CHARACTER

INT 21 - DOS 1+ - WRITE CHARACTER TO STANDARD OUTPUT

- 1 AH = 02h
- 2 DL = character to write

Return: AL = last character output

As used at the beginning of this book, it prints a single character represented by the number in DL. It can be seen as the equivalent of C's "putchar".

D-2109-WRITE STRING

INT 21 - DOS 1+ - WRITE STRING TO STANDARD OUTPUT

- 1 AH = 09h
- 2 DS:DX -> '\$'-terminated string

Return: AL = 24h (the '\$' terminating the string)

This function is weird. It prints a string until it finds a dollar sign. This was a way that strings were terminated before the convention of zero terminators like in C or C++ became common. You can save a few bytes by terminating your strings with \$ instead of zero. My putstring method expects zero because I follow the modern convention.

D-2139-MKDIR

INT 21 - DOS 2+ - "MKDIR" - CREATE SUBDIRECTORY

- 1 AH = 39h
- 2 DS:DX -> ASCIZ pathname

Return: CF clear if successful AX destroyed CF set on error AX = error code (03h,05h) (see #01680 at AH=59h/BX=0000h)

I am not sure why someone would create a directory inside an assembly program since it could be done before the program is run and also included in a zip file if someone distributes their programs to other people, but this system call is most likely how DOS's mkdir command is implemented because it is an important thing to do!

D-213A-RMDIR

INT 21 - DOS 2+ - "RMDIR" - REMOVE SUBDIRECTORY

- 1 AH = 3Ah
- 2 DS:DX -> ASCIZ pathname of directory to be removed

Return: CF clear if successful AX destroyed CF set on error AX = error code (03h,05h,06h,10h) (see #01680 at AH=59h/BX=0000h)

Notes: directory must be empty (contain only '.' and '..' entries)

D-213B-CHDIR

INT 21 - DOS 2+ - "CHDIR" - SET CURRENT DIRECTORY

- 1 AH = 3Bh
- 2 DS:DX -> ASCIZ pathname to become current directory
- 3 (max 64 bytes)

Return: CF clear if successful AX destroyed CF set on error AX = error code (03h) (see #01680 at AH=59h/BX=0000h)

Notes: if new directory name includes a drive letter, the default drive is not changed, only the current directory on that drive

D-213C-CREAT

INT 21 - DOS 2+ - "CREAT" - CREATE OR TRUNCATE FILE

- 1 AH = 3Ch
- 2 CX = file attributes (see #01401)
- 3 DS:DX -> ASCIZ filename

Return: CF clear if successful AX = file handle CF set on error AX = error code (03h,04h,05h) (see #01680 at AH=59h/BX=0000h)

Notes: if a file with the given name exists, it is truncated to zero length

Bitfields for file attributes: Bit(s) Description (Table 01401) 0 read-only 1 hidden 2 system 3 volume label (ignored) 4 reserved, must be zero (directory) 5 archive bit 7 if set, file is shareable under Novell NetWare

D-213D-OPEN

INT 21 - DOS 2+ - "OPEN" - OPEN EXISTING FILE

- 1 AH = 3Dh
- 2 AL = access and sharing modes (see #01402)
- 3 DS:DX -> ASCIZ filename
- 4 CL = attribute mask of files to look for (server call only)

Return: CF clear if successful AX = file handle CF set on error AX = error code (01h,02h,03h,04h,05h,0Ch,56h) (see #01680 at AH=59h)

Bitfields for access and sharing modes:

Table 01402

Bit(s) Description 2-0 access mode

- 000 read only
- 001 write only
- 010 read/write

D-213E-CLOSE

INT 21 - DOS 2+ - "CLOSE" - CLOSE FILE

- 1 AH = 3Eh
- 2 BX = file handle

Return: CF clear if successful AX destroyed CF set on error AX = error code (06h) (see #01680 at AH=59h/BX=0000h)

Notes: if the file was written to, any pending disk writes are performed, the time and date stamps are set to the current time, and the directory entry is updated

D-213F-READ

INT 21 - DOS 2+ - "READ" - READ FROM FILE OR DEVICE

```
1 AH = 3Fh
2 BX = file handle
3 CX = number of bytes to read
4 DS:DX -> buffer for data
```

Return: CF clear if successful AX = number of bytes actually read (0 if at EOF before call) CF set on error AX = error code (05h,06h) (see #01680 at AH=59h/BX=0000h)

Notes: data is read beginning at current file position, and the file position is updated after a successful read the returned AX may be smaller than the request in CX if a partial read occurred

D-2140-WRITE

INT 21 - DOS 2+ - "WRITE" - WRITE TO FILE OR DEVICE

```
1 AH = 40h
2 BX = file handle
3 CX = number of bytes to write
4 DS:DX -> data to write
```

Return: CF clear if successful AX = number of bytes actually written CF set on error AX = error code (05h,06h) (see #01680 at AH=59h/BX=0000h)

Notes: if CX is zero, no data is written, and the file is truncated or extended to the current position data is written beginning at the current file position, and the file position is updated after a successful write

D-2141-UNLINK

INT 21 - DOS 2+ - "UNLINK" - DELETE FILE

```
1 AH = 41h
2 DS:DX -> ASCIZ filename (no wildcards, but see notes)
3 CL = attribute mask for deletion (server call only, see notes)
```

Return: CF clear if successful AX destroyed (DOS 3.3) AL seems to be drive of deleted file CF set on error AX = error code (02h,03h,05h) (see #01680 at AH=59h/BX=0000h)

Notes: (DOS 3.1+) wildcards are allowed if invoked via AX=5D00h, in which case the filespec must be canonical (as returned by AH=60h), and only files matching the attribute mask in CL are deleted. DR DOS 5.0-6.0 returns error code 03h if invoked via AX=5D00h; DR DOS 3.41 crashes if called via AX=5D00h with wildcards. DOS does not erase the file's data; it merely becomes inaccessible.

D-2142-LSEEK

INT 21 - DOS 2+ - "LSEEK" - SET CURRENT FILE POSITION

```
1  AH = 42h
2  AL = origin of move
3      00h start of file
4      01h current file position
5      02h end of file
6  BX = file handle
7  CX:DX = (signed) offset from origin of new file position
```

Return: CF clear if successful. DX:AX = new file position in bytes from start of file. CF set on error. AX = error code (01h, 06h) (see #01680 at AH=59h/BX=0000h).

Notes: for origins 01h and 02h, the pointer may be positioned before the start of the file; no error is returned in that case (except under Windows NT), but subsequent attempts at I/O will produce errors if the new position is beyond the current end of file, the file will be extended by the next write (see AH=40h);

D-214300-GET FILE ATTRIBUTES

INT 21 - DOS 2+ - GET FILE ATTRIBUTES

```
1  AX = 4300h
2  DS:DX -> ASCIZ filename
```

Return: CF clear if successful. CX = file attributes (see #01420). AX = CX (DR DOS 5.0). CF set on error. AX = error code (01h, 02h, 03h, 05h) (see #01680 at

AH=59h) Notes: under the FlashTek X-32 DOS extender, the filename pointer is in DS:EDX under DR DOS 3.41 and 5.0, attempts to change the subdirectory bit are simply ignored without an error BUG: Windows for Workgroups returns error code 05h (access denied) instead of error code 02h (file not found) when attempting to get the attributes of a nonexistent file. This causes open() with O_CREAT and fopen() with the “w” mode to fail in Borland C++. SeeAlso: AX=4301h,AX=4310h,AX=7143h,AH=B6h,INT 2F/AX=110Fh,INT 60/DI=0517h

D-214301-CHMOD

INT 21 - DOS 2+ - “CHMOD” - SET FILE ATTRIBUTES

```
1  AX = 4301h
2  CX = new file attributes (see #01420)
3  DS:DX -> ASCIZ filename
```

Return: CF clear if successful AX destroyed CF set on error AX = error code (01h,02h,03h,05h) (see #01680 at AH=59h)

Notes: will not change volume label or directory attribute bits, but will change the other attribute bits of a directory (the directory bit must be cleared to successfully change the other attributes of a directory, but the directory will not be changed to a normal file as a result) MS-DOS 4.01 reportedly closes the file if it is currently open for security reasons, the Novell NetWare execute-only bit can never be cleared; the file must be deleted and recreated under the FlashTek X-32 DOS extender, the filename pointer is in DS:EDX DOS 5.0 SHARE will close the file if it is currently open in sharing-compatibility mode, otherwise a sharing violation critical error is generated if the file is currently open DR DOS 3.41/5.0 will silently ignore attempts to change the ‘directory’ attribute bit SeeAlso: AX=4300h,AX=4311h,AX=7143h,INT 2F/AX=110Eh

Bitfields for file attributes: Bit(s) Description (Table 01420) 7 shareable (Novell NetWare) 7 pending deleted files (Novell DOS, OpenDOS) 6 unused 5 archive 4 directory 3 volume label execute-only (Novell NetWare) 2 system 1 hidden 0 read-only

D-214C-EXIT

INT 21 - DOS 2+ - “EXIT” - TERMINATE WITH RETURN CODE

- 1 AH = 4Ch
- 2 AL = return code

Return: never returns

Notes: unless the process is its own parent (see #01378 [offset 16h] at AH=26h), all open files are closed and all memory belonging to the process is freed all network file locks should be removed before calling this function

SeeAlso: AH=00h,AH=26h,AH=4Bh,AH=4Dh,INT 15/AH=12h/BH=02h,INT 20,INT 22 SeeAlso: INT 60/DI=0601h

D-2159-GET ERROR INFO

INT 21 - DOS 3.0+ - GET EXTENDED ERROR INFORMATION

- 1 AH = 59h
- 2 BX = 0000h

Return: AX = extended error code (see #01680) BH = error class (see #01682) BL = recommended action (see #01683) CH = error locus (see #01684) ES:DI may be pointer (see #01681, #01680) CL, DX, SI, BP, and DS destroyed

Notes: functions available under DOS 2.x map the true DOS 3.0+ error code into one supported under DOS 2.x you should call this function to retrieve the true error code when an FCB or DOS 2.x call returns an error under DR DOS 5.0, this function does not use any of the DOS-internal stacks and may thus be called at any time

SeeAlso: AH=59h/BX=0001h,AX=5D0Ah,INT 2F/AX=122Dh,INT 24

Table 01680

Values for DOS extended error code:

- 00h (0) no error
- 01h (1) function number invalid
- 02h (2) file not found
- 03h (3) path not found

- 04h (4) too many open files (no handles available)
- 05h (5) access denied
- 06h (6) invalid handle
- 07h (7) memory control block destroyed
- 08h (8) insufficient memory
- 09h (9) memory block address invalid
- 0Ah (10) environment invalid (usually >32K in length)
- 0Bh (11) format invalid
- 0Ch (12) access code invalid
- 0Dh (13) data invalid
- 0Eh (14) reserved
- 0Eh (14) (PTS-DOS 6.51+, S/DOS 1.0+) fixup overflow
- 0Fh (15) invalid drive
- 10h (16) attempted to remove current directory
- 11h (17) not same device
- 12h (18) no more files
- 13h (19) disk write-protected
- 14h (20) unknown unit
- 15h (21) drive not ready
- 16h (22) unknown command
- 17h (23) data error (CRC)
- 18h (24) bad request structure length
- 19h (25) seek error
- 1Ah (26) unknown media type (non-DOS disk)
- 1Bh (27) sector not found
- 1Ch (28) printer out of paper
- 1Dh (29) write fault
- 1Eh (30) read fault
- 1Fh (31) general failure
- 20h (32) sharing violation
- 21h (33) lock violation
- 22h (34) disk change invalid (ES:DI -> media ID structure)(see #01681)
- 23h (35) FCB unavailable
- 23h (35) (PTS-DOS 6.51+, S/DOS 1.0+) bad FAT
- 24h (36) sharing buffer overflow
- 25h (37) (DOS 4.0+) code page mismatch
- 26h (38) (DOS 4.0+) cannot complete file operation (EOF / out of input)
- 27h (39) (DOS 4.0+) insufficient disk space
- 28h-31h reserved

D-2162-GET PSP ADDRESS

INT 21 - DOS 3.0+ - GET CURRENT PSP ADDRESS

1 AH = 62h

Return: BX = segment of PSP for current process

Notes: this function does not use any of the DOS-internal stacks and may thus be called at any time, even during another INT 21h call the current PSP is not necessarily the caller's PSP identical to the undocumented AH=51h SeeAlso: AH=50h,AH=51h